The Journal of Systems and Software 112 (2016) 11–25

Contents lists available at ScienceDirect

The Journal of Systems and Software

journal homepage: www.elsevier.com/locate/jss

Thermal-throttling server: A thermal-aware real-time task scheduling framework for three-dimensional multicore chips

Ting-Hao Tsai, Ya-Shu Chen*

Department of Electrical Engineering, National Taiwan University of Science and Technology, Taipei City, Taiwan, ROC

ARTICLE INFO

Article history:
Received 20 March 2015
Revised 1 October 2015
Accepted 27 October 2015
Available online 9 November 2015

Keywords:
Thermal-aware
Real-time task scheduling
Three-dimensional multicore chip

ABSTRACT

Three-dimensional (3D) multicore chips have been recently developed to deal with the power consumption and interconnection delay problems of embedded systems; however, thermal management has proven to be challenging due to the heat effect of vertically stacked cores, and the subsequent trade-off that occurs between performance requirements and overheating. In this paper we propose a novel thermal-aware real-time scheduling framework for 3D multicore chips to achieve an effective trade-off between system temperature and task schedulability for dynamic workloads. A thermal-throttling server is first proposed to adjust the heat generated by task executions, and a thermal-throttling dispatcher is then presented to enable thermal-awareness in well-known real-time dispatchers. An admission control is subsequently derived to ensure that all task executions satisfy the thermal and timing constraints. Lastly, a series of extensive simulations are carried out, with encouraging results in terms of schedulability and the prevention of overheating.

© 2015 Elsevier Inc. All rights reserved.

1. Introduction

The Multicore System-on-a-Chip design has been widely used in embedded systems due to the high computational requirements of many modern applications. As the number of cores and components in a system increases, so do the power consumption and global interconnect delay to a significant degree. Three-dimensional (3D) stack-based homogeneous multicore designs (Healy et al., 2010; Dreslinski et al., 2013) have been developed to increase transistor density and to improve performance through vertically stacking multiple silicon layers connected by through silicon vias (TSVs) (Cong et al., 2011). The stacked structure, however, causes a higher power density and poorer heat dissipation, which exacerbates the problem of thermal hot spots. Higher on-chip temperatures impact reliability and degrade performance. As such, the trade-off between system performance and temperature has become a critical problem for 3D multicore chips.

Numerous studies have explored thermal-aware scheduling in 3D multicore chips (Coskun et al., 2009; Hameed et al., 2011; Kang et al., 2011; Li et al., 2013; Liu et al., 2010; Lung et al., 2011; Tsai and Chen, 2012; Zhou et al., 2010a; Liao et al., 2015). Ignoring timing constraints, some studies (Coskun et al., 2009; Liu et al., 2010; Zhou et al., 2010a) have utilized thermal-aware task allocations to balance the peak temperatures of each core in order to minimize the peak temperature; others (Hameed et al., 2011; Lung et al., 2011) have proposed algorithms to minimize the number of hot spots or maximize the throughput. Some studies (Kang et al., 2011; Liao et al., 2015) have applied dynamic voltage scaling to satisfy the thermal constraint. Li et al. (2013) considered the timing constraint for the dependence tasks and proposed a rotation scheduling to minimize the peak temperature. Tsai and Chen (2012) proposed an algorithm to maximize the system utilization under a thermal constraint. To reduce the product size and total cost of the embedded system, thermal constraints would be satisfied using thermal-aware task scheduling instead of additional cooling devices. Liao et al. (2015) assigned tasks according to the nondecreasing order of the peak power and applied the dynamic voltage frequency scaling (DVFS) technique to cores by considering the thermal constraint. For 2D multicore chips, some studies (Chen et al., 2009a; Fisher et al., 2009; Huang et al., 2014) proposed thermal-aware scheduling under thermal/timing constraint(s), and Quan and Chaturvedi (2010) proposed a scheduling utilization bound that took thermal factors into account. Different from the thermal behavior in 2D multicore chips, the thermal behavior in 3D multicore chips needs to take account of the thermal conduction in the vertically aligned cores. The thermal conduction between horizontally adjacent cores is equalization (i.e., the temperature among cores is balanced because heat is dissipated from a high temperature core to a low temperature core); however, the thermal conduction between vertically aligned cores leads to heat accumulation (i.e., the temperature of the core in the upper layer cannot be lower than that of the vertically aligned core in the lower layer). It results in more complicated task dispatching for trade-off the system temperature and

* Corresponding author. Tel.: +886 27376702.
E-mail address: d10007401@mail.ntust.edu.tw, yschen@mail.ntust.edu.tw (Y.-S. Chen).

http://dx.doi.org/10.1016/j.jss.2015.10.038
0164-1212/© 2015 Elsevier Inc. All rights reserved.

utilization. Very few thermal-aware real-time scheduling studies of 3D multicore chips have focused on system schedulability in light of the peak temperature constraint.

To investigate the schedulability in the presence of the peak temperature constraint in 3D multicore chips, a partition-based thermal-throttling server was proposed in Tsai and Chen (2014). In contrast to the hardware throttling mechanism, a thermal-throttling server manages the thermal budget of each core by reserving appropriate cooling utilization without violating the timing constraint. In this study, we extended the thermal-throttling server to a global dispatcher and also devised a method to determine the thermal size of each core for meeting the run-time thermal constraint. Admission control is also presented for a given task set to improve the system robustness. The thermal management capabilities of global and partition schemas were evaluated in the absence admission control, and the derived schedulability of admission control under global and partition schemas and different thermal size assignments were compared to provide system designers with further insight.

This study is motivated by a desire to investigate the complexities raised by the trade-off that exists between system performance and temperature in 3D multicore chips. Our contributions are as follows:

- A novel thermal-aware multicore real-time scheduling framework is proposed to dynamically manage the run-time system temperature and provide a quality of service guarantee for 3D stacked multicore chips.
- To deal with the thermal conduction from vertically aligned cores in 3D multicore chips, thermal size assignment is proposed to manage the heat generated by task executions. The thermal management effect on system performance is also evaluated.
- In contrast to off-line optimization approaches, a thermal-throttling server is proposed to determine and change the core's execution frequency for run-time temperature management based on the given thermal size and incoming task information.
- A thermal-throttling dispatcher is presented as an extension of well-known real-time dispatchers, i.e., First-Fit and Global, for run-time thermal-aware task dispatching and for dynamically switching the thermal size assignment policy with dynamic workload.
- The thermal-aware admission control is proposed to ensure that all tasks meet the timing and thermal constraints, from the perspective of single task, single core, and a task set with multicore, enabling thermal awareness in all real-time schedulers.

The rest of this paper is organized as follows: Section 2 presents the system model; Section 3 proposes a thermal-aware real-time task scheduling framework for a 3D multicore system, including thermal size assignment, thermal-throttling server, thermal-throttling dispatcher, and admission control; Section 4 details the performance evaluation; Section 5 presents our conclusions.

2. System model

This study considers a 3D homogenous multicore chip, as shown in Fig. 1, and the chip consists of a set of active silicon layers $\mathbf{L} = \{L_1, L_2, \ldots, L_h\}$. Each active silicon layer has one or more cores, and a thermal interface material (TIM) is used between each silicon layer to provide efficient heat transfer. The vertically aligned cores are modeled as a *Stack*, and they communicate each other by TSVs. In order to remove heat, one side of the chip is soldered to the heat sink. The *bottom layer* closest to the heat sink is L_1, and the cores on L_1 are *bottom cores*. The *top layer* is the layer farthest from the heat sink, and the cores located in this layer are *top cores* (Zhou et al., 2010a).

The proposed system consists of a set of independent periodic tasks $\mathbf{\Gamma} = \{\tau_1, \tau_2, \ldots, \tau_m\}$, where m is the number of tasks in $\mathbf{\Gamma}$. Each task τ_i in the system is associated with a worst-case execution core

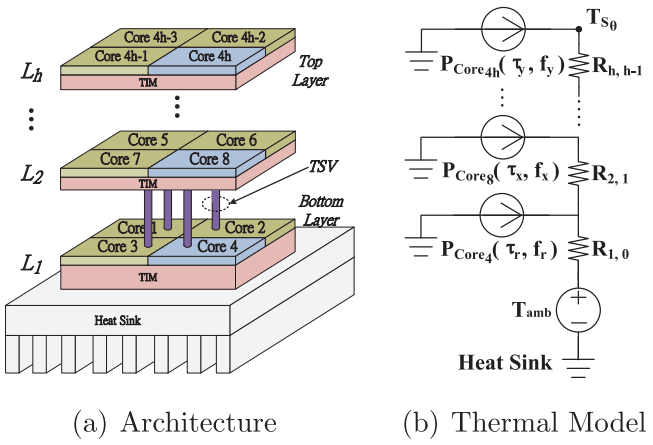


Fig. 1. 3D multicore chip.

cycle c_i (Wilhelm et al., 2008), a power consumption $P_{Core_j}(\tau_i, f_{max})$, and a relative deadline (period) d_i. The utilization u_i of a task τ_i is the ratio of the worst-case execution time to the relative deadline, when the task is executed at the maximum frequency of the system (i.e., $\frac{c_i}{f_{max} \times d_i}$)(Liu and Layland, 1973). The number of core cycles executed in a time interval depends on the core frequency. In other words, the computation time for a task τ_i execution at the frequency f_i is $\frac{c_i}{f_i}$.

Each core $Core_j$ in the system has the same computing resources and an independent dynamic voltage/frequency scaling (DVFS) capabilities. The power consumption function of each core running a task τ_i can be divided into dynamic power and leakage power consumptions, $P_{Core_j}(\tau_i, f_i) = P_{dyn,j}(\tau_i, f_i) + P_{leak,j}$. The dynamic power consumption $P_{dyn,j}(\tau_i, f_i)$ can be modeled as a convex function of the core frequency (Rabaey et al., 2002): $P_{Core_j}(\tau_i, f_i) = \frac{1}{2} C_{ef,\tau_i} V_{dd}^2 f_i$, where C_{ef,τ_i}, V_{dd}, and κ denote the effective switch capacitance when executing the task τ_i, the supply voltage, and the hardware-design-specific constant, respectively. The above equation shows that $P \propto f_i^{\rho}$ (Zhu, 2006; Xu et al., 2005); we assume that ρ is 2 in this paper,[1] and assumes that the core can operate at any frequency f_i in a given range $[f_{min}, f_{max}]$, where f_{min} is larger than the lowest frequency required to eliminate the effect of leakage power consumption (Jejurikar et al., 2004). Further extensions of this work for processors with discrete available frequencies can be done using approaches similar to those reported in Huang et al. (2014), Chaturvedi et al. (2010). The leakage power consumption $P_{leak,j}$ can be modeled as $P_{leak,j} = \hbar + \varsigma T_j$, where T_j is the absolute temperature on $Core_j$ and $\hbar$ and ς both are constants (Chen et al., 2009b; Liao et al., 2005; Chantem et al., 2011; Liu et al., 2007).

In a single core, according to Fourier heat flow analysis and the RC model (Skadron et al., 2002), the approximate temperature of each $Core_j$ at time t can be estimated as:

$$T_{Core_j}(t) = P_{Core_j}(\tau_i, f_i) \times R_j + T_{amb} + (T_{init} - (P_{Core_j}(\tau_i, f_i) \times R_j + T_{amb}))e^{-t/R_j C_j} \quad (1)$$

where T_{init} is the initial temperature of the core, $P_{Core_j}(\tau_i, f_i)$, R_j, C_j are the power consumption, the thermal resistance, and the thermal capacitance of $Core_j$. According to Eq. (1), the heat of $Core_j$ caused by a task τ_i at a frequency f_i is no larger than $P_{Core_j}(\tau_i, f_i) \times R_j$. The accuracy of this function depends on the accuracy of the power consumption estimate. In this study, the peak power was considered to avoid system overheating.

In a 3D multicore chip, thermal coupling of the vertically adjacent cores leads to a larger thermal influence among them compared with

[1] The proposed approach is also applied for varying the value of ρ (e.g., $\rho \geq 2$).

horizontally adjacent cores. On the basis of the thermal evaluation report in Zhu et al. (2008), Zhou et al. (2010b), Jin et al. (2015), Kang et al. (2011), Lee et al. (2015), Kang et al. (2014), Yang et al. (2006), the thermal model of a 3D multicore chip can be simplified by focusing only on the heat flow between vertically adjacent cores. In this study, to meet the thermal constraint, we considered the peak temperature among horizontally adjacent cores in each layer and managed the peak temperature of each stack with vertically adjacent cores, as shown in Fig. 1(b).[2] When a core $Core_j$ in a 3D homogenous multicore chip is considered, as shown in Fig. 1(b), the thermal resistance R_j of each core $Core_j$ shall take thermal conduction from the vertically aligned cores and from the TSVs into account (Cong et al., 2011). This study assumes that the thermal resistance of cores located in the same layer is the same due to homogenous floorplanning (Zhu et al., 2008). Assuming the thermal resistance between layer L_v and layer L_{v-1} is $R_{v,v-1}$, the thermal resistance R_j of a core $Core_j$ allocated at layer L_q is calculated as Eq. (2).

$$R_j = \sum_{v=1}^{q} R_{v,v-1} \tag{2}$$

Based on Eqs. (1) and (2), the maximum heat of each core $Core_j$ caused by a task τ_i executing at frequency f_i is calculated as Eq. (3) below, where R_j is the thermal resistance with thermal conduction considered:

$$\triangle T_{Core_j} = P_{Core_j}(\tau_i, f_i) \times R_j \tag{3}$$

Following the thermal model as shown above and in Fig. 1(b), the peak temperature of the system is the maximum temperature among the stacks, and the temperature of a stack S_θ can be calculated by Eq. (4), and expressed as a value in absolute temperature with units K.

$$T_{S_\theta} = T_{amb} + \sum_{\forall Core_j \in S_\theta} \triangle T_{Core_j} \tag{4}$$

Given a set of real-time tasks Γ and a 3D homogeneous multicore chip, there are three technical issues that must be addressed in this paper: (1) how to manage the heat generated from task execution, (2) how to schedule and dispatch tasks while considering the timing and thermal constraints simultaneously, and (3) how to derive the admission control to guarantee task execution under the timing and thermal constraints.

3. Thermal-throttling framework for 3D multicore chips

3.1. Framework

In this study, we explore a thermal-aware real-time task scheduling framework for 3D multicore chips. In contrast to optimization approaches, the concept of a thermal-throttling server is proposed to confer thermal awareness to existing real-time task scheduling. The rationale behind the concept is a trade-off between thermal management and utilization management. A thermal-throttling server based on on-line scheduling framework is shown in Fig. 2. During run-time, in order to guarantee the thermal constraint, the thermal-throttling dispatcher assigns thermal size (described in Section 3.3) to each core according to the current system temperature. For each run-time arrival task, the task is allocated to a proper core by the used dispatcher (Section 3.5) when it has passed the admission control (Section 3.6). To improve schedulability of on-line tasks, the proposed thermal-throttling dispatcher can dynamically switches the policy of thermal size assignment. After a task is dispatched to a core, the task is scheduled by the corresponding thermal-throttling server (Section 3.4)

[2] More details on the thermal management of 2D multicore chips for managing horizontal heat flow are provided in Mizunuma et al. (2013), Yun et al. (2013), Chou et al. (2013), Chu et al. (2015), and the thermal management concept can be extended to the thermal-throttling server for accurate temperature prediction.

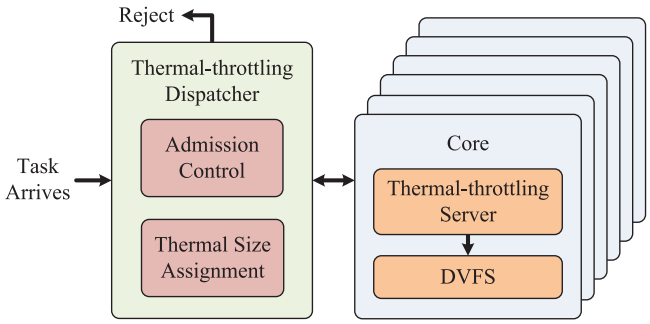


Fig. 2. Framework for thermal-aware real-time task scheduling.

with dynamic voltage frequency scaling for meeting the thermal constraint and timing constraint. Each thermal-throttling server feeds back the current frequency assignment to the thermal-throttling dispatcher, and then the dispatcher may adjust the thermal size of the cores to improve the schedulability bound of the admission control based on the current workload.

3.2. Motivating example

In this section, an example is presented to show the side effects of traditional real-time task scheduling without thermal management, and to present the idea of the thermal-throttling dispatcher and server. The process will also show the concept of admission control to guarantee that tasks meet the timing and thermal constraints.

Suppose there is a task set, Γ, containing five independent tasks and scheduled by earliest-deadline-first (EDF) algorithm. In order to simplify this example, each task has two parameters: peak power and utilization, e.g., $\tau_i : (P_{Core_j}(\tau_i, f_{max}), u_i)$. The characteristics of each task are: $\Gamma = \{\tau_1 : (70, 0.8), \tau_2 : (70, 0.4), \tau_3 : (60, 0.3), \tau_4 : (50, 0.2), \tau_5 : (40, 0.2)\}$. The hardware architecture of a three-dimensional chip is a one-stack and two-layer system. $Core_1$ and $Core_2$ are located in the bottom layer and the top layer, respectively. The thermal resistances of $Core_1$ and $Core_2$ are 0.54 K/W and 0.79 K/W, respectively. The ambient temperature is 318 K, and the thermal constraint is 358 K. To meet the thermal constraint, the total heat resulting from $Core_1$ and $Core_2$ shall not be larger than 40 K.

Based on the first-fit algorithm, the given task set is partitioned into cores, as shown in Fig. 3(a). That is, τ_1 and τ_4 are partitioned into $Core_1$, and the total utilization is 1, while τ_2, τ_3, and τ_5 are partitioned into $Core_2$, and the total utilization is 0.9. When any tasks are executed in $Core_1$ and $Core_2$, e.g., τ_4 and τ_5, at the same time, and then according to Eq. (4), the temperature of $Core_2$ will violate the thermal constraint, i.e., $318\,\text{K} + 50\,\text{W} \times 0.54\,\frac{\text{K}}{\text{W}} + 40\,\text{W} \times 0.79\,\frac{\text{K}}{\text{W}} = 376.6\,\text{K} > 358\,\text{K}$. Since $Core_1$ is fully utilized, the system cannot lower the core frequency for reducing the temperature by dynamic voltage frequency scaling to cool the system.

To meet the timing and thermal constraints, we propose the concept of a thermal-throttling dispatcher and server. The thermal-throttling dispatcher is responsible for dispatching tasks and assigning the thermal size to each core. Each core has a thermal-throttling server with the assigned thermal size, and all allocated tasks are scheduled by the thermal-throttling server. Based on the current ambient temperature T_{amb} and the thermal constraint T_c, the total heat generated by task executions shall be no larger than $TS_{max} = T_c - T_{amb}$, where TS_{max} is defined as the total thermal size. In this example, the derived total thermal size in the system is 40 K, i.e., 358 K − 318 K. According to Eq. (3) and the given assigned thermal size, the tolerable peak power and utilization of each core can be derived as shown in Fig. 4. Unlike in the 2D multicore chip, the heat effect of vertically stacked cores shall be considered. The core nearest to the heat sink, which has smaller resistance, will generate less

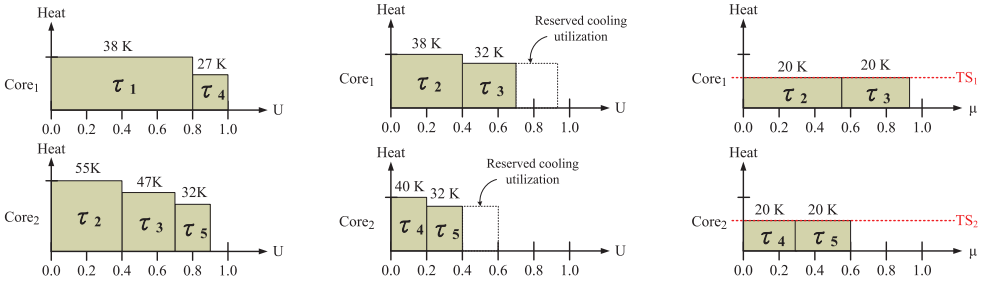


Fig. 3. First-fit versus thermal-throttling server with first-fit.

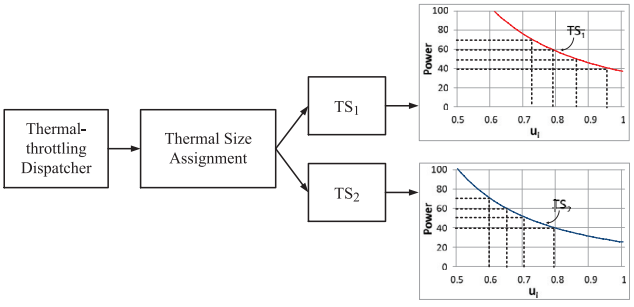


Fig. 4. Thermal size assignment.

heat. When the thermal-throttling dispatcher sequentially assigns the same thermal size to thermal-throttling servers on $Core_1$ and $Core_2$, e.g., $TS_1 = TS_2 = 20\,\text{K}$, then the tolerable peak power and utilization of each core will not be the same due to different values of the thermal resistance. For example, when the utilization bound of a task is set as 0.8, the tolerable peak powers of a task on $Core_1$ and $Core_2$ are approximately 60 W and 40 W, respectively.

In contrast to the traditional real-time dispatcher, the thermal-throttling dispatcher dispatches tasks with both timing and thermal constraints considered. Instead of fully utilizing the cores, the dispatcher partitions tasks with cooling utilization reserved to accept high peak power task execution under the thermal constraint. Given a thermal size of 20 K, if $Core_1$ and $Core_2$ execute a task with a peak power of 70 W, then the total utilization of $Core_1$ and $Core_2$ shall be no more than 0.73 and 0.6, respectively. Otherwise, the system overheats, since the peak power and utilization of τ_1 are 70 W and 0.8, respectively. Thus, τ_1 is not feasible for either $Core_1$ or $Core_2$. In order to meet the thermal constraint, our proposed thermal-throttling dispatcher will not dispatch τ_1 to any core. Although the peak power of τ_2 is 70 W, τ_2 is feasible for $Core_1$ and $Core_2$ because the utilization of τ_2 is 0.4 (which is not greater than 0.73 or 0.6). Here, we still dispatch tasks using the first-fit algorithm. As shown in Fig. 3(b), τ_2 and τ_3 are partitioned into $Core_1$. The peak power and the total utilization of $Core_1$ are 70 W and 0.7, respectively. Thus, τ_4 and τ_5 are partitioned into $Core_2$. The peak power and the total utilization of $Core_2$ are 50 W and 0.4, respectively.

As shown in Fig. 3(b), when any tasks are executed in $Core_1$ and $Core_2$, e.g., τ_2 and τ_4, at the same time, then, according to Eq. (4), the temperature of $Core_2$ will still violate the thermal constraint, i.e., $318\,\text{K} + 70\,\text{W} \times 0.54\,\frac{\text{K}}{\text{W}} + 50\,\text{W} \times 0.79\,\frac{\text{K}}{\text{W}} = 395.3\,\text{K} > 358\,\text{K}$. After tasks are partitioned, in order to meet the thermal constraint, the thermal-throttling server is invoked to switch the execution frequency for reclaiming cooling utilization. Without violating the thermal constraint, when a task τ_i is assigned to $Core_j$ with thermal size TS_j, the feasible frequency f_i can be calculated by:

$$P_{Core_j}(\tau_i, f_i) = \frac{TS_j}{R_j} \tag{5}$$

where $P_{Core_j}(\tau_i, f_i)$ is the power of the core that executes task τ_i at frequency f_i, and R_j is the thermal resistance of $Core_j$. This equation is derived according to Eq. (3). To meet the timing constraint, the lengthened execution time shall be less than the task's deadline, i.e., $\frac{c_i}{f_i \times d_i} \leq 1$. In this paper, the utilization u_i denotes the ratio of the worst-case execution time to the relative deadline, when the task is executed at the maximum frequency of the system (i.e., $\frac{c_i}{f_{max} \times d_i}$). The lengthened utilization μ_i of each task τ_i, when task τ_i is executed at a lower frequency f_i, can be calculated as:

$$\mu_i = u_i \times \frac{f_{max}}{f_i} \leq 1 \tag{6}$$

Based on the above equations, the feasible frequencies of τ_2 and τ_3 are 0.73 and 0.79 for $Core_1$, respectively. The lengthened utilizations of τ_2 and τ_3 are 0.55 and 0.38 for $Core_1$, respectively. After lowering the execution frequency, the total utilization of $Core_1$ and $Core_2$ are lengthened to 0.93 and 0.6, respectively. Because the total utilization is less than 1, all tasks can meet the timing constraint. According to Eq. (3), the total heat generated by task executions on $Core_1$ and $Core_2$ are both equal to 20 K, so the system can meet the thermal constraint, i.e., $318\,\text{K} + 20\,\text{K} + 20\,\text{K} \leq 358\,\text{K}$, as shown in Fig. 3(c). The guaranteed timing and thermal constraints are verified in Section 3.3.

3.3. Thermal size assignment

In this section, the thermal size is presented to manage the heat generated from task execution for each core. Given the thermal constraint T_c and the ambient temperature T_{amb}, the tolerable heat generated by task executions is $T_c - T_{amb}$. To deal with the thermal effect from vertically stacked cores, the total thermal size for all vertically stacked cores can be defined as:

$$TS_{max} = \sum_{i=1}^{h} TS_i = T_c - T_{amb} \tag{7}$$

where h is the total number of layers. Given the total thermal size TS_{max}, the problem of assigning thermal size to each vertically stacked core for maximizing the system utilization under different task sets is an NP-hard problem. This problem can be reduced to the subset sum problem (Chrobak et al., 2011). Compared to the thermal effect from vertically stacked cores, the thermal effect from horizontally aligned cores is relatively smaller (Zhou et al., 2010a; Li et al., 2013). Nonetheless, to eliminate the thermal effect from horizontally aligned cores, all horizontally aligned cores that are in the same layer are assigned the same thermal size. In this section, we discuss how to efficiently assign the thermal size to each layer. Three heuristics of thermal size assignment, termed average, balanced and proportional, are presented.

The intuitive approach, called average assignment (*Avg*), is to averagely assign the total thermal size to each layer, such as:

$$TS_1 = TS_2 = \cdots = TS_h = \frac{TS_{max}}{h} \tag{8}$$

where h is the total number of layers. The thermal resistances of each pair of vertically aligned cores are not the same, as shown in Fig. 1(b). This results in different tolerable peak powers and utilizations in different layers even with the same thermal size values. Under *Avg*, the tolerable peak power and utilization of the cores near the heat sink are higher than those of the cores far away the heat sink. The high power tasks shall be sequentially assigned to the cores near the heat sink and be executed in the dedicated core, when *Avg* is used. In other words, if tasks are randomly assigned or migrated between cores, then the *Avg* might perform worse.

To derive the same tolerable peak power and utilization for each layer, the thermal size is assigned to each layer according to the corresponding thermal resistance, an approach called balanced assignment (*Ba*). Following Eqs. (2) and (4), the thermal size can be calculated by:

$$TS_q = TS_{max} \times \frac{R_q}{R_1 + R_2 + \cdots + R_h} \tag{9}$$

where q is the index of the layer number. With identical tolerable power consumption in each layer, the task can be dispatched to any core without considering thermal-aware placement. When tasks are migrated between cores during run-time, such as in a global dispatcher, *Ba* can accept more task executions than *Avg*, because the smallest thermal size in the system under *Ba* is higher than that under *Avg*. More evaluations are shown in Section 4.

When the total thermal size is small due to the strict thermal constraint, a non-equal tolerable peak power and non-equal thermal size assignment might be needed to improve task feasibility. A possible approach is to assign the larger thermal size to the layer near to the heat sink, called proportional assignment (*Pro*). Following Eqs. (2) and (4), the thermal size can be assigned to each layer proportionally according to:

$$TS_q = TS_{max} \times \frac{R_q^{-1}}{R_1^{-1} + R_2^{-1} + \cdots + R_h^{-1}} \tag{10}$$

By doing so, tasks assigned to the layer near to the heat sink can have higher power and higher utilization, but tasks assigned to the layer far away from the heat sink can only have lower power and lower utilization. Similar to *Avg*, with non-equal tolerable peak power for each layer, tasks shall be assigned to the cores and be executed in the dedicated core with consideration of the task's peak power under *Pro*. Compared to *Avg*, *Pro* is more suitable for a task set with a large variation of peak powers and utilizations for the tasks, owing to the larger thermal size of the layer near to the heat sink. The time complexity of *Avg*, *Ba*, and *Pro* are $O(hn)$, $O(h^2n)$, and $O(h^2n)$, respectively. The thermal-throttling dispatcher can switch these thermal size assignments during run-time. Based on the proposed idea, smart heuristics can be proposed to better match the given workload. It should be noted that the thermal size assignment problem is an NP-hard problem. The main objective of this paper is to propose a thermal-aware real-time task scheduling framework.

3.4. Thermal-throttling server

To schedule tasks while considering the timing and thermal constraints simultaneously, we propose a novel concept, the **thermal-throttling server**, in this section. In each core, there is a thermal-throttling server, which can be implemented at the operating system level with an integrated dynamic voltage frequency scaling (DVFS) technique, for independently switching the execution frequency (Qualcomm Snapdragon Processor). The aim of a thermal-throttling server is to execute tasks with proper frequencies according to the current thermal budget.

The details of the thermal-throttling server operations are shown in Algorithm 1. After assigning the thermal size to each core by the specific approaches shown in Section 3.3, the thermal budget of a server is replenished with the assigned thermal size. When the system is initialized, a new job of a task is scheduled, or the preempted task resumes. The frequency of the core is switched to the feasible frequency[3,4] that can meet the timing constraint of the scheduled/resumed task and can meet the thermal size constraint, as shown in Eqs. (5) and (6). The available frequency of a task is chosen by considering the peak power of the task without violating the thermal constraint. Because scaling down the frequency will increase the execution time, the increased utilization of a task is considered to meet the timing constraint. The server informs the dispatcher that the task is schedulable and switches the frequency for task execution. Otherwise, if the frequency is unable to be determined, then the server informs the dispatcher that the task is unschedulable. Note that the utilization bound $U(m_j)$ of EDF scheduler is 1. When a task is executed, the thermal budget of the thermal-throttling server is

Algorithm 1 Thermal-throttling Server.

I Budget Replenishing:

- Thermal-throttling server replenishes the thermal budget with the assigned thermal size
  - when the system is initialized.
  - when a new job of a task is scheduled.
  - when the preempted task resumes.

II Frequency Determining:

- When the thermal budget is replenished, thermal-throttling server calculates the feasible frequency according the current thermal budget, such that
  - $P_{Core_j}(\tau_i, f_i) \leq \frac{TS_j}{R_j}$
  - $f_{min} \leq f_i \leq f_{max}$
  - $u_i \times \frac{f_{max}}{f_i} \leq 1$
  - $\mu_{Core_j} + u_i \times \frac{f_{max}}{f_i} \leq U(m_j)$, when a partition schema dispatcher is used
- If there is a feasible execution frequency f_i, τ_i is accepted.
- If there is a dispatcher in the system, returns the frequency f_i and the acceptance/refusal of task τ_i to the dispatcher.

III Frequency Switching:

- After the frequency was determined, the frequency of the core is switched to the feasible frequency.

IV Budget Consuming:

- When the frequency is switched, the corresponding thermal budget is also consumed.

V Task Scheduling:

- All tasks are scheduled by EDF scheduling.

[3] If the discrete frequency is considered, the largest frequency which is no larger than feasible frequency would be chosen for satisfying the thermal constraint. For example, if the feasible frequency is 0.8 and the available frequencies are {1, 0.7, 0.4}, the frequency of the core is switched to 0.7.

[4] In this paper, we assume that the frequency switching overhead is relatively smaller (Zhou et al., 2010a). When the frequency switch overhead is significant, the feasible frequency estimation could take the switch overhead into account.

Algorithm 2 TS-FF.

Input: A 3D multicore chip with hn cores, a task set Γ, the hot task power threshold P_{th}, and the total thermal size TS_{max}.

1: **if** System is initializing **then**
2: **for** each $j \leftarrow 1 : hn$ **do**
3: $\mu_{Core_j} \leftarrow 0$
4: Assign the thermal size TS_j
5: Initialize the thermal-throttling server with TS_j
6: **for** each task τ_i in Γ **do**
7: **if** the peak power of $\tau_i \geq P_{th}$ **then**
8: $j = 1$
9: **while** $j \leq hn$ **do**
10: Assign τ_i to $Core_j$
11: Invoke Algorithm 1 to handle the events
12: **if** τ_i is accepted **then**
13: Move task τ_i from Γ to Γ_j
14: $\mu_{Core_j} \leftarrow \mu_{Core_j} + u_i \times \frac{f_{max}}{f_i}$
15: Break
16: $j = j + 1$
17: **else**
18: $j = hn$
19: **while** $j \geq 1$ **do**
20: Assign τ_i to $Core_j$
21: Invoke Algorithm 1 to handle the events
22: **if** τ_i is accepted **then**
23: Move task τ_i from Γ to Γ_j
24: $\mu_{Core_j} \leftarrow \mu_{Core_j} + u_i \times \frac{f_{max}}{f_i}$
25: Break
26: $j = j - 1$
27: **if** τ_i is unfeasible for all cores **then**
28: Remove task τ_i from Γ
29: **while** System is running **do**
30: Each thermal-throttling server handles all scheduling events according to Algorithm 1.

consumed. Because the execution power of an idle task is ignored in this study, the thermal budget is not consumed/replenished when the system is idle. During run-time, each task is scheduled according to EDF scheduling. The time complexity of the thermal-throttling server is $O(m_j)$, where m_j is the number of tasks in $Core_j$.[5]

3.5. Thermal-throttling dispatcher

To dispatch tasks with consideration of the timing and thermal constraint simultaneously, in this section, the partition and global dispatchers are extended with the concept of the thermal-throttling server, referred to as Thermal-throttling First-Fit (*TS-FF*) and Thermal-throttling Global (*TS-G*), as shown in Algorithm 2 and Algorithm 3, respectively. The concept of a thermal-throttling dispatcher to render thermal awareness to other real-time dispatchers with dynamic workload is then proposed in Algorithm 4. Note that the thermal-throttling dispatcher also can be implemented with Worst-Fit, Next-Fit, etc.

To maximize the system utilization under the thermal constraint, we assign the arrival tasks with higher peak power (i.e., hot tasks) to a core that is near to the heat sink. The tasks with lower peak power (i.e., cool tasks) are assigned to a core that is far from the heat sink, following the idea presented in Tsai and Chen (2012), Zhou et al. (2010a). Note that, in order to reduce the time complexity in searching, tasks under consideration are sorted and indexed in non-decreasing order of their peak power, and cores with small indexes are nearer to the heat sink. In this way, the peak power of each stack can be better balanced.

[5] The code size might need several *kB* for applying the thermal-throttling server into the existing scheduler.

Algorithm 3 TS-G.

Input: A 3D multicore chip with hn cores and the total thermal size TS_{max}.

1: **if** System is initializing **then**
2: **for** each $j \leftarrow 1 : hn$ **do**
3: Assign the thermal size TS_j
4: Initialize the thermal-throttling server with TS_j
5: **while** System is running **do**
6: **if** A task is ready or a task is finished **then**
7: Find the highest priority ready task τ_i
8: **for** each $j \leftarrow 1 : hn$ **do**
9: **if** $Core_j$ is idle **then**
10: Assign τ_i to $Core_j$
11: Invoke Algorithm 1 to handle the events
12: **if** τ_i is accepted **then**
13: Break
14: Find the core $Core_k$ which is executing the lowest priority task among all cores
15: **if** The priority of $\tau_i >$ the priority of the current task in $Core_k$ **then**
16: Assign τ_i to $Core_k$
17: Invoke Algorithm 1 to handle the events
18: **if** τ_i is accepted **then**
19: Break
20: **else**
21: τ_i is waiting in the ready queue until a task is finished
22: **else**
23: Reject task τ_i

Algorithm 4 Thermal-throttling Dispatcher.

I Thermal Size Assignment:

- Assign thermal size to each core.
  - when the system is initialized.
  - when a new task is added into the workload.
  - when a task is rejected by the thermal-throttling server.

II Task Dispatching:

- Invoke admission control (optional).
- Assign tasks to cores according to the given dispatcher.
- Invoke the corresponding thermal-throttling server to handle the events.

In Algorithm 2, when the system is initializing, the current utilization, μ_{Core_j}, of each core is set to 0 and the thermal size is set according to a specific policy, as shown in Section 3.3. The thermal-throttling server in each core is then initialized using the given thermal size (Steps 1–5). For each hot (cool) task, we search the proper allocatable core from the bottom (top) layer to the top (bottom) layer (Steps 7–9 and 17–19). To meet the thermal constraint, the thermal-throttling server is invoked to determine the execution frequency f_i through the task-level admission test (Lemma 1) and core-level admission test (Lemma 2). If task τ_i passes the admission control, then it is assigned to $Core_j$ (Steps 10–15 and 20–25). The execution time is increased by a lower execution frequency, so the task utilization is recalculated as the ratio of increased execution time for the task to

its deadline, as in Eq. (6). If the sum of the utilization of $Core_j$ and the lengthened utilization for task τ_i is no larger than the utilization bound of the given scheduler, i.e., EDF, then task τ_i is allocated to $Core_j$ and the total utilization of $Core_j$ is updated (Steps 14 and 24). The task feasibility is verified in Section 3.6. If task τ_i fails to be assigned to any core, it is removed from the task set (Steps 27 and 28). When the system is running, the thermal-throttling server in each core handles all scheduling events according to Algorithm 1 (Steps 29 and 30). The time complexity of Algorithm 2 is $O(hnm)$, where hn is the number of cores and m is the number of tasks. All the partitioned tasks are fixedly executed by corresponding cores, so the partition schema is invoked only when the system is initializing or when there is a new task added into the system. When a new task τ_i is added into the task set Γ during run-time, the partition schema (Steps 6–28) can be invoked again to assign task τ_i to a dedicated core.

In contrast to partition schema dispatching the tasks into each local queue, the global schema dispatches all tasks into a global queue and schedules the highest priority task in the global queue. A task can be migrated from one core to another core and may be preempted by the highest priority task. When the preempted task resumes, it may be assigned into different core (Zapata and Alvarez, 2005). Based on the concept of global schema, we applied the thermal-throttling server into Global EDF as TS-G as illustrated in Algorithm 3. When the system is initializing, the thermal size assignment is triggered and the thermal-throttling servers are initialized (Steps 1–4). When a task τ_i is ready, the scheduler first searches the core in an idle state, and it invokes the mechanisms in Algorithm 1 to handle the corresponding events (Steps 6–11). At the same time, the current task with the lowest priority among all cores is recorded (Step 14). Once all cores are busy, the current executed task in $Core_k$ with the lowest priority is compared with the arrival task τ_i; if τ_i has higher priority, then the mechanisms are invoked in Algorithm 1 (Steps 15–17). If there is no proper core to schedule task τ_i immediately, then the task is kept waiting in the ready queue until a task is finished. (Step 21). The time complexity of Algorithm 3 is $O(hnm)$, where hn is the number of cores and m is the number of tasks. The correctness of the task feasibility is proved in Section 3.6.

As we discussed in Section 3.3, the thermal size assignment is an NP-complete problem. The aims of Algorithms 2 and 3 are to assign thermal sizes when the system is initialized for reduced run-time overhead. When the dynamic workload is considered for the system, the main operations of the thermal-throttling dispatcher are shown in Algorithm 4. The thermal size can be assigned when the system is initialized, and then the given dispatcher is as shown in Algorithms 2 and 3. The admission control shown in Theorem 1 can either be invoked for guaranteeing the system robustness or not be invoked for better run-time schedulability. When a new task is added into the system or a task is rejected by the thermal-throttling server, the thermal size can be reassigned for better schedulability bound to serve more tasks. After assigning thermal size, similar to the operations in Algorithms 2 and 3, all tasks are assigned to a proper core by the given dispatcher, and scheduled by the corresponding thermal-throttling server under a feasible frequency. All real-time dispatchers can be extended to become thermal-throttling dispatchers based on Algorithm 4.

3.6. Admission control

To guarantee that the given task set meets the thermal and timing constraints under a 3D multicore chip, a fast admission control is proposed in this section. Three feasibility tests, for a single task under a single core, a task set under a single core, and a task set under a 3D multicore chip, are presented.

Lemma 1. *Given a core $Core_j$ with thermal size TS_j, thermal resistance R_j, and thermal constraint T_c, a task τ_i is feasible with the execution frequency f_i under core $Core_j$, if*

$$P_{Core_j}(\tau_i, f_i) \times R_j \le TS_j \le T_c - T_{amb},$$

$$u_i \times \frac{f_{max}}{f_i} \le 1, \text{ and}$$

$$f_{min} \le f_i \le f_{max},$$

where u_i is the utilization of task τ_i when τ_i is executed at the maximum frequency f_{max}, and T_{amb} is the ambient temperature.

Proof. The correctness of this lemma follows from two facts:

(1) Given a thermal constraint T_c and the ambient temperature T_{amb}, the tolerable heat generated by task executions is $T_c - T_{amb}$. To meet the thermal constraint, the tolerable heat generated by the core $Core_j$, which is the thermal size TS_j, shall be no larger than $T_c - T_{amb}$. Based on the RC model (Skadron et al., 2002), the heat generated by task τ_i is $P_{Core_j}(\tau_i, f_i) \times R_j$, as shown in Eq. (3). The task τ_i meets the thermal constraint, if the heat generated by task τ_i is no larger than the tolerable heat of the core $Core_j$, i.e., TS_j.

(2) In this study, the task utilization u_i is the ratio of the worst-case execution time to the relative deadline, when the task is executed at the maximum frequency of the system, i.e., $u_i = \frac{c_i}{f_{max} \times d_i}$. When a task is executed at a frequency f_i, which is no larger than the maximum frequency f_{max} of the core and no less than the minimum frequency f_{min} of the core, the lengthened utilization of τ_i is $\frac{c_i}{f_i \times d_i} = u_i \times \frac{f_{max}}{f_i}$. The task τ_i meets the timing constraint if the lengthened utilization is no larger than 1 (Yao et al., 1995).

Before a task is assigned into a dedicated core, Lemma 1 is used to check the feasibility of the given task. To assign a task set to the proper core under a 3D multicore chip, the feasibility of the given task set Γ_j under the core $Core_j$ can be tested by Lemma 2.

Lemma 2. *Given a core $Core_j$ with thermal size TS_j, thermal resistance R_j, and thermal constraint T_c, the task set Γ_j is feasible under core $Core_j$ if $\forall \tau_i \in \Gamma_j$:*

$$P_{Core_j}(\tau_i, f_i) \times R_j \le TS_j \le T_c - T_{amb},$$

$$\sum_{\forall \tau_i \in \Gamma_j} u_i \times \frac{f_{max}}{f_i} \le U(m_j), \text{ and}$$

$$f_{min} \le f_i \le f_{max},$$

where u_i is the utilization of task τ_i when τ_i is executed at the maximum frequency f_{max}, f_i is the execution frequency of τ_i, m_j is the number of tasks in the task set Γ_j, $U(m_j)$ is the utilization bound of the given scheduler with m_j tasks, and T_{amb} is the ambient temperature.

Proof. The correctness of this lemma follows from two facts:

(1) Suppose there are two tasks, τ_x and τ_y, in the core $Core_j$. At time $t = t_1$, the scheduled task is switched from τ_x to τ_y and the temperature $T_{Core_j}(t_1)$ results from executing task τ_x. According to Eq. (1) and based on the RC model (Skadron et al., 2002), and assuming $T_{init} = T_{amb}$, the peak temperature of the core $Core_j$ by executing task τ_x is $P_{Core_j}(\tau_x, f_x) \times R_j + T_{amb}$. Thus, $T_{Core_j}(t_1)$ is no larger than $P_{Core_j}(\tau_x, f_x) \times R_j + T_{amb}$. Assuming that at time $t = t_1 + t_2$ the task τ_y is completed, the resulting temperature is $T_{Core_j}(t_1 + t_2)$. When $T_{init} = T_{Core_j}(t_1)$, the thermal behavior of the core can be estimated by considering the following three cases:

Case 1: $T_{init} > P_{Core_j}(\tau_y, f_y) \times R_j + T_{amb}$: According to Eq. (1), the temperature decreases exponentially, and the peak temperature of core $Core_j$ during the time interval $[t_1, t_1 + t_2]$ is $T_{Core_j}(t_1)$. Following the above statement, $T_{Core_j}(t_1)$ is no larger than $P_{Core_j}(\tau_x, f_x) \times R_j + T_{amb}$.

Case 2: $T_{init} = P_{Core_j}(\tau_y, f_y) \times R_j + T_{amb}$: During the time interval $[t_1, t_1 + t_2]$, the temperature is at a constant value. Thus, the peak

temperature of core $Core_j$ is bounded by $T_{Core_j}(t_1+t_2)$, which is no larger than $P_{Core_j}(\tau_y, f_y) \times R_j + T_{amb}$.

Case 3: $T_{init} < P_{Core_j}(\tau_y, f_y) \times R_j + T_{amb}$: During the time interval $[t_1, t_1+t_2]$, according to Eq. (1), the temperature is increased as $P_{Core_j}(\tau_y, f_y) \times R_j + T_{amb} + (T_{init} - (P_{Core_j}(\tau_y, f_y) \times R_j + T_{amb}))e^{-t_2/R_jC_j}$. Because $T_{init} < P_{Core_j}(\tau_y, f_y) \times R_j + T_{amb}$, the peak temperature of core $Core_j$ is no larger than $P_{Core_j}(\tau_y, f_y) \times R_j + T_{amb}$. As shown in the above cases, the peak temperature of core $Core_j$ is bounded by the $\max(P_{Core_j}(\tau_i, f_i) \times R_j) + T_{amb}$ among all tasks τ_i executed in $Core_j$. The task set Γ_j meets the thermal constraint if the heat generated by each task τ_i, i.e., $P_{Core_j}(\tau_i, f_i) \times R_j$, in Γ_j is no larger than the TS_j and TS_j is no larger than $T_c - T_{amb}$.

(2) Following the correctness of Lemma 1, the lengthened utilization of each task τ_i in Γ_j is $u_i \times \frac{f_{max}}{f_i}$, when a task is executed in a frequency f_i, which is no larger than the maximum frequency f_{max} of the core and no less than the minimum frequency f_{min} of the core. According to the proofs in Aydin et al. (2001), the task set Γ_j meets the timing constraint, if the total lengthened utilization of all tasks in Γ_j, i.e., $\sum_{\forall \tau_i \in \Gamma_j} u_i \times \frac{f_{max}}{f_i}$, is no larger than the utilization bound $U(m_j)$ of the given scheduler, e.g., the utilization bound $U(m_j)$ of EDF is 1.

After assigning the thermal size to each core, the feasibility test for a task set under the given 3D multicore chips is given in Theorem 1.

Theorem 1. *Given a 3D multicore chip C with h layers, each layer containing n cores, and the thermal constraint T_c, a periodic task set Γ is feasible under the 3D multicore chip C if $\forall Core_j \in C, \forall \tau_i \in \Gamma$:*

$$P_{Core_j}(\tau_i, f_i) \times R_j \le \min\{TS_1, \ldots, TS_j, \ldots, TS_{hn}\},$$

$$\sum\{TS_j | \forall Core_j \in C \text{ and vertically aligned}\} \le T_c - T_{amb},$$

$$\sum_{\forall \tau_i \in \Gamma} u_i \times \frac{f_{max}}{f_i} \le U(m, hn, \alpha), \text{ and}$$

$$f_{min} \le f_i \le f_{max},$$

where TS_j and R_j are the thermal size and thermal resistance of core $Core_j$, f_i is the execution frequency of τ_i, u_i is the utilization of task τ_i when τ_i is executed at the maximum frequency f_{max}, and $U(m, hn, \alpha)$ is the utilization bound of the given dispatcher and scheduler,[6] where α is $\max\{u_i \times \frac{f_{max}}{f_i}\}$ among tasks τ_i in Γ. with m tasks and hn cores, α is the maximum lengthened task utilization in Γ, and T_{amb} is the ambient temperature.

Proof. The correctness of this theorem follows from two facts: (1) Following the correctness of Lemma 1, the lengthened utilization of each task τ_i in Γ is $u_i \times \frac{f_{max}}{f_i}$. Without considering the communication overhead, a 3D multicore with h layers and each layer containing n cores can be taken as a multicore chip with hn cores. According to the proofs in [26], the task set Γ with m tasks meets the timing constraint under the EDF scheduler with the given dispatcher in hn cores, if the total lengthened utilization of all tasks in Γ, i.e., $\sum_{\forall \tau_i \in \Gamma} u_i \times \frac{f_{max}}{f_i}$, is no larger than the utilization bound $U(m, hn, \alpha)$ of the given dispatcher and scheduler,[7] where α is $\max\{u_i \times \frac{f_{max}}{f_i}\}$ among tasks τ_i in Γ.

(2) The feasibility of tasks under the thermal constraint can be proved by mathematical induction on the number of cores in each layer, n, and the number of layers, h. By the definition in Eq. (3), $\triangle T_{Core_j} = P_{Core_j}(\tau_i, f_i) \times R_j$. To simplify the description, $P_{Core_j}(\tau_i, f_i) \times R_j$ are represented by $\triangle T_{Core_j}$ in the following statement.

Case 1: Single core

Consider $n = 1, h = 1$, a chip C containing 1 layer and 1 core. Then, the correctness of the theorem is shown by fact 1 and Lemma 2.

Case 2: Horizontally aligned cores

Induction base: Consider $n = 2, h = 1$, a chip C containing 1 layer and 2 cores, $Core_1$ and $Core_2$, and that the task τ_i is the task with the highest power in the task set. When τ_i is executed by $Core_1$ and $Core_2$ is executing the task with the second highest power, there are two conditions for the generated heat: (1) $\triangle \mathbf{T_{Core_1}} = \triangle \mathbf{T_{Core_2}}$: According to the RC thermal model and the second law of thermodynamics, there is no thermal conduction between these two cores. If $\triangle T_{Core_1}$ and $\triangle T_{Core_2}$ both hold for $min\{TS_1, TS_2\}$, and TS_1 and TS_2 are both no larger than $T_c - T_{amb}$, then the system does not overheat in this case, because $\max\{\triangle T_{Core_1}, \triangle T_{Core_2}\} + T_{amb} \le min\{TS_1, TS_2\} + T_{amb} \le T_c$. (2) $\triangle \mathbf{T_{Core_1}} \ne \triangle \mathbf{T_{Core_2}}$: There is thermal conduction, and $\triangle T_{Core_1}$ is the largest heat value because $Core_1$ executes τ_i. According to Eq. (4), the highest temperature is $\triangle T_{Core_1} + T_{amb}$. If $\triangle T_{Core_1} \le min\{TS_1, TS_2\}$, and TS_1 and TS_2 are both no larger than $T_c - T_{amb}$, then the system does not overheat in this case. This is because $\max\{\triangle T_{Core_1}, \triangle T_{Core_2}\} + T_{amb} = \triangle T_{Core_1} + T_{amb} \le min\{TS_1, TS_2\} + T_{amb} \le T_c$. Based on the above cases, a task set Γ meets the thermal constraint, if TS_1 and TS_2 are both no larger than $T_c - T_{amb}$ and the heat generated by τ_i, $\triangle T_{Core_j} = P_{Core_j}(\tau_i, f_i) \times R_j$, on any core is no larger than either TS_1 or TS_2.

Induction hypothesis: Assume that this theorem holds, when there is 1 layer and $k-1$ horizontally aligned cores in the chip C.

Induction step: Consider $n = k, h = 1$, a chip C that contains 1 layer and k cores, and that the task τ_i having the highest power in the task set Γ is executed by $Core_x$ in the core set C, and that the other cores are respectively executing the highest power tasks in the corresponding task sets of their local queues. The resulting thermal effect can be analyzed in two conditions: (1)$\triangle \mathbf{T_{Core_1}} = \triangle \mathbf{T_{Core_2}} = \cdots = \triangle \mathbf{T_{Core_k}}$: The theorem holds and the correctness can be shown by following the proof of Condition (1) in the Induction base. (2)$\triangle \mathbf{T_{Core_1}} \ne \triangle \mathbf{T_{Core_2}} \ne \cdots \ne \triangle \mathbf{T_{Core_k}}$: The theorem holds and the correctness can be shown by following the proof of Condition (2) in the Induction base.

If the cores are not executing the highest power tasks at the same time, then the system temperature is no larger than the worst-case, as shown above.

Case 3: Vertically aligned cores in a single stack

Induction base: Consider $n = 1, h = 2$, a chip C containing 2 layers and 2 vertically aligned cores, where $Core_1$ and $Core_2$ are respectively allocated in L_1 and L_2, in the stack S_1, and that the task τ_i is the task with the highest power in the task set Γ. There are two conditions when τ_i is assigned to one of the cores. (1) τ_i is executed by $Core_2$ and $Core_1$ is executing the task with the second highest power in the task set Γ. According to Eqs. (2) and (3), as $Core_2$ is allocated in L_2, which has the largest thermal resistance, $Core_2$ generates the largest heat, i.e., $\triangle T_{Core_2}$, in the core set. With the cores considered to be vertically aligned, the temperature of the system is calculated by Eq. (4), i.e., $\triangle T_{Core_1} + \triangle T_{Core_2} + T_{amb}$, and the accumulated thermal size is $TS_1 + TS_2$. If the largest heat, $\max\{\triangle T_{Core_1}, \triangle T_{Core_2}\} = \triangle T_{Core_2}$, holds for the smallest thermal size, $min\{TS_1, TS_2\}$, and the accumulated thermal size is bounded by the thermal constraint, $TS_1 + TS_2 \le T_c - T_{amb}$, then the system does not overheat as $\triangle T_{Core_1} + \triangle T_{Core_2} + T_{amb} \le TS_1 + TS_2 + T_{amb} \le T_c$. (2) τ_i is executed by $Core_1$ and $Core_2$ is executing the task with the second highest power. $Core_1$ may not generate the largest heat as $Core_1$ is allocated in L_1, which has the smallest thermal resistance. Similarly, if the largest heat, $\max\{\triangle T_{Core_1}, \triangle T_{Core_2}\}$, holds for the smallest thermal size, $min\{TS_1, TS_2\}$, and the accumulated thermal size is bounded by the thermal constraint, $TS_1 + TS_2 \le T_c - T_{amb}$, then the system does not overheat. Based on the above conditions, a task set Γ meets the thermal

[6] The utilization bound of EDF First-Fit and Global EDF are $U^{\text{EDF-FF}}(m, hn, \alpha) \le \frac{\beta(hn)+1}{\beta+1}$ and $U^{\text{G-EDF}}(m, hn, \alpha) \le (hn)(1-\alpha)+\alpha$, respectively, where β is $\lfloor \frac{1}{\alpha} \rfloor$(López et al., 2000).

[7] As shown in [27], the utilization bound of EDF First-Fit and Global EDF are $U^{\text{EDF-FF}}(m, hn, \alpha) \le \frac{\beta(hn)+1}{\beta+1}$ and $U^{\text{G-EDF}}(m, hn, \alpha) \le (hn)(1-\alpha)+\alpha$, where β is $\lfloor \frac{1}{\alpha} \rfloor$.

constraint if $TS_1 + TS_2 + T_{amb}$ is no larger than T_c and the heat generated by any core when it executes τ_i holds for both TS_1 and TS_2.

Induction hypothesis: Assume that this theorem holds when there are $q-1$ layers and $q-1$ vertically aligned cores in the chip C.

Induction step: Consider $n = 1$, $h = q$, a chip C that contains q layers and q vertically aligned cores, and that each $Core_j$ is allocated in L_j in the stack S_1. The task τ_i is the task with the highest power in the task set Γ. There are two conditions when τ_i is assigned to one of the cores. (1) τ_i is executed by $Core_x$, which is allocated in the top layer L_q, and the other cores are respectively executing the highest power tasks in the corresponding task sets of their local queues. According to Eqs. (2) and (3), the largest heat is generated by $Core_x$, $\triangle T_{Core_x}$, as $Core_x$ has the largest thermal resistance and the task with the highest power. When considering vertically aligned cores, the system temperature is calculated as $\sum_{j=1}^{q} \triangle T_{Core_j} + T_{amb}$, following Eq. (4), and the accumulated thermal size is $\sum_{j=1}^{q} TS_j$. If $\max_{\forall Core_j \in C}\{\triangle T_{Core_j}\} = \triangle T_{Core_x}$ can hold for $\min\{TS_1, TS_2, \ldots, TS_q\}$, and the accumulated thermal size is bounded by the thermal constraint, $\sum_{j=1}^{q} TS_j \le T_c - T_{amb}$, then the system does not overheat. This is because $\sum_{j=1}^{q} \triangle T_{Core_j} + T_{amb} \le \sum_{j=1}^{q} TS_j + T_{amb} \le T_c$. (2) τ_i is executed by $Core_y$ which is not allocated in the top layer L_q, and the other cores are respectively executing the highest power tasks in the corresponding task sets of their local queues. $Core_y$ may not generate the largest heat, because $Core_y$ does not have the largest thermal resistance. Let the core that generates the largest heat in the system be $Core_x$; then, the theorem holds following the statements shown in Condition (1). Based on the above conditions, the theorem holds.

Case 4: Vertically aligned cores with a multi-stack

Induction base: Consider $n = 2$, $h = 2$, a chip C containing 2 layers and 4 cores, and that L_1 contains $Core_1$ and $Core_2$ and L_2 contains $Core_3$ and $Core_4$. $Core_1$ and $Core_3$ are allocated in stack S_1 and $Core_2$ and $Core_4$ are allocated in stack S_2. The task τ_i is the task with the highest power in the task set Γ. There are two conditions when τ_i is assigned to one of the cores. (1) τ_i is executed by the core, $Core_3$ or $Core_4$, that is allocated in the top layer L_2. Based on the inferences from Cases 2 and 3, if $\max\{\triangle T_{Core_3}, \triangle T_{Core_4}\}$ is no larger than $\min\{TS_1, TS_2, TS_3, TS_4\}$, and for each stack the accumulated thermal sizes, $TS_1 + TS_3$ and $TS_2 + TS_4$, are bounded by the thermal constraint $T_c - T_{amb}$, then the system does not overheat. (2) τ_i is executed by the core, $Core_1$ or $Core_2$, that is not allocated in the top layer L_2. Based on the inferences from Cases 2 and 3, if $\max\{\triangle T_{Core_1}, \triangle T_{Core_2}, \triangle T_{Core_3}, \triangle T_{Core_4}\}$ is no larger than $min\{TS_1, TS_2, TS_3, TS_4\}$, and for each stack the accumulated thermal sizes, $TS_1 + TS_3$ and $TS_2 + TS_4$, are bounded by $T_c - T_{amb}$, then the system does not overheat. Based on the above conditions, the theorem holds.

Induction hypothesis: Consider $n = k$, $h = q-1$, and assume that this theorem holds.

Induction step: Consider $n = k$, $h = q$, and a chip C containing q layers and k cores in each layer. The task τ_i is the task with the highest power in the task set Γ. There are two conditions when τ_i is assigned to one of the cores. (1) τ_i is executed by the core, $Core_x$, that is allocated in the top layer L_q. Based on the inferences from Cases 2 and 3, the system does not overheat if $\max_{\forall Core_j \in C}\{\triangle T_{Core_j}\} = \triangle T_{Core_x}$ holds for the thermal size $min_{\forall Core_j \in C}\{TS_j\}$, and, by considering the vertically aligned cores in each stack, the accumulated thermal size $\sum_{\forall Core_j \in C}$ and vertically aligned TS_j is bounded by $T_c - T_{amb}$. (2) τ_i is executed by the core that is not allocated in the top layer L_q. Let the core that generates the largest heat in the system be $Core_x$; thus, the proof is similar to the above statements.

The correctness of the theorem can be shown by following the above considerations. A task set Γ never violates the thermal constraint, if the accumulated thermal size of the vertically aligned cores for each stack is bounded by the difference between the thermal constraint and the ambient temperature, $\sum_{\forall Core_j \in C}$ and vertically aligned $TS_j \le T_c - T_{amb}$, and the heat generated by each core when it executes τ_i holds for the smallest thermal size, $P_{Core_j}(\tau_i, f_i) \times R_j = \triangle T_{Core_j} \le \min(TS_1, \ldots, TS_j, \ldots, TS_{hn}), \forall Core_j \in C$. A task set Γ never violates the timing constraint if the lengthened utilization of each task τ_i in Γ_j is no larger than the utilization bound of the given dispatcher and scheduler, $\sum_{\forall \tau_i \in \Gamma} u_i \times \frac{f_{max}}{f_i} \le U(m, hn, \alpha)$ and $f_{min} \le f_i \le f_{max}$.

As mentioned in Section 3.5, to meet the timing constraint and thermal constraint simultaneously, tasks are dispatched when they pass the admission control. When the partition schema is used, tasks can be tested by Lemmas 1 and 2 to decide the assigned core and feasible frequency for a better schedulability bound, since Theorem 1 provides the sufficient condition. As tasks are migrated between cores under the global schema, to guarantee the feasibility of tasks during run-time, the task set shall be tested by Theorem 1. More evaluation results for dispatchers with different admission control are compared in Section 4.

The proposed thermal-throttling server can also be extended with a Rate Monotonic (RM) scheduler. When the RM scheduler is used, the timing constraint of each admission control shown above will be replaced by the utilization bound of the RM scheduler (Liu and Layland, 1973). For example, $U(m_j)$ and $U(m, hn, \alpha)$ as shown in Lemma 2 and Theorem 1 can be applied as $U(m_j) = m_j(2^{1/m_j} - 1)$ and $U^{\text{RM-FF}}(m, hn, \alpha) = (hn-1)(2^{\frac{1}{\beta+1}} - 1)\beta + (m - \beta(hn-1))(2^{\frac{1}{m-\beta(hn-1)}} - 1)$, where $\beta = \lfloor \frac{1}{\log_2(\alpha+1)} \rfloor$, under Rate-Monotonic First-Fit scheduling (Lopez et al., 2003), respectively.

4. Performance evaluation

This section presents the performance of the proposed approaches and comparisons. Section 4.1 introduces the experiment setting and the simulation environment. To evaluate the run-time system capacity, Section 4.2.1 compares the thermal size assignments under partition and global schemas without admission control consideration. Section 4.2.2 compares the schedulability derived from admission control under partition and global schemas with varied thermal size assignment. Section 4.2.3 evaluates the capability of thermal management by comparing it with related works. Section 4.2.4 details the peak temperature and reserved cooling utilization of each core to show the properties of the thermal-throttling server.

4.1. Data sets and performance metrics

In this section, we evaluate the performance of the proposed ideas regarding their scheduling and temperature management capabilities.

The experiment evaluates the performance of *TS-FF* (*TS-FF(AD)*), as shown in Algorithm 2, and that of *TS-G* (*TS-G(AD)*), as shown in Algorithm 3, *BS+DVFS* (Zhou et al., 2010a), and *VGVS* (Liao et al., 2015). In *TS-FF(AD)*, admission control (Theorem 1) is introduced in Algorithm 2 (before Step 6), and only tasks passing admission control during run-time are scheduled. In *TS-G(AD)*, admission control (Theorem 1) is incorporated in Algorithm 3 (before Step 8), and the task set is schedulable when it passes admission control. The comparison Temperature-Balancing-by-Stack (*BS*), which balances the power consumption of each core to minimize the peak temperature, is implemented to swap the workloads of the hottest stack and the coolest stack every 500 cycles and reclaim the idle slack for reducing the execution frequency as *BS+DVFS*. The other comparison Vertical-Group-Voltage-Scaling (*VGVS*) assigns tasks to cores according to the peak power of the cores and then assigns a feasible voltage (frequency) according to the peak power of the cores to satisfy the thermal constraint. The core near the heat sink is assigned a high voltage because the core shows high heat distribution efficiency. By contrast, a low

Table 1
Benchmark trace.

Benchmark	Peak power (W)	Peak temp. (K)	Benchmark	Peak power (W)	Peak temp. (K)
art	57.75	358.62	fma3d	36.66	341.96
parsar	57.51	358.43	equake	36.03	341.46
vortex	56.60	357.71	galgel	35.73	341.23
ammp	54.37	355.95	apsi	35.45	341.01
gcc	52.39	354.39	swim	34.09	339.93
vpr	52.37	354.37	bzip2	33.73	339.65
gzip	51.01	353.3	lucas	33.19	339.22

Table 2
Thermal characteristic.

Parameter	Value	Unit
TIM thermal conductivity	4	W/(m K)
TIM thermal capacitance	4.0×10^6	J/(m^3 K)
Cu thermal conductivity	400	W/(m K)
Cu thermal capacitance	3.55×10^6	J/(m^3 K)
Si thermal conductivity	100	W/(m K)
Si thermal capacitance	1.75×10^6	J/(m^3 K)

voltage is assigned to the core far from the heat sink. In *TS-FF, TS-FF(AD), TS-G*, and *TS-G(AD)*, tasks are executed at the assigned frequency as per Algorithm 1.

Similar to Hameed et al. (2011), Kang et al. (2014), Jin et al. (2015), this study measured the performance results by using the power trace of the SPEC2K suite. The power characteristics of each core on a 3D homogeneous multicore chip were identical, and PTScalar (PTS, 2003) (which incorporated the power evaluation function into SimpleScalar (Austin et al., 2002)) was used as the performance/power simulator to estimate the power of benchmarks for Alpha 21264 with $V_{dd} = 1.3$ V and an execution frequency of 3 GHz. The output power trace could then be used as the input data for HotSpot (2010). The temperature of execution of each benchmark for a core was estimated using HotSpot (2010) for 2*M* cycles, as shown in Table 1. The power threshold was set as 50 W because the peak temperature was greater than 353 K. In the following experiment, a task with a peak power greater than 50 W was a hot task. Otherwise, the task was a cool task. As shown in Table 1, *gcc* is a hot task and *bzip*2 is a cool task. On the basis of evaluated power patterns, 100 randomly generated task sets were measured to average out the errors; each set consisted of 100 tasks. The periods of the tasks were uniformly distributed in the interval [30, 2000], and the execution times were uniformly distributed in the interval [1, 1000]. The utilization range of the tasks was set as 0.01 to 0.1 to avoid the side effects of an unbalanced task partition. The power of each task was in the ranges [50, 60] and [30, 40] according to the SPEC2K suite. The thermal constraint, ambient temperature, and maximum thermal size were 358 K, 318 K, and 40 K, respectively.

The following experiments evaluate 3D multicore chips with 2 and 4 layers and 4 Alpha 21264 microprocessor cores per layer, and each core has a size of 4.56×4.56 mm^2. The thickness of the top layer is 20 μm, the thickness of the bottom layer is 300 μm, and the thickness of the thermal interface material (TIM) is 20 μm. The thermal characteristics of the hardware platform are estimated by HotSpot (2010) and are listed in Table 2, where Cu and Si are copper and silicon, respectively.

In the following experiments, five performance metrics were considered: the achieved ratio, miss ratio, thermal impact ratio (TIR), peak temperature (PT), and scheduled utilization. The *Achieved Ratio* is the ratio of the worst-case utilization that the system can achieve without violating the timing constraints to the total utilization of the task set. The *Miss Ratio* is the ratio of the number of schedulable task sets to the total number of test task sets. Let *X* and *Y* denote the duration of thermal constraint violation in all cores and the hyperperiod of the system workload, respectively. The *thermal impact ratio (TIR)* of the evaluated algorithm is *X*/*Y*. The *peak temperature (PT)* represents the PT of the core for the given task set. *Utilization* represents the average worst-case utilization that the system can achieve without violating the timing constraints for the input task sets.

4.2. Experimental results

4.2.1. Comparison of thermal size assignments in the absence of admission control

This section discusses the experiment on the run-time schedulability of the thermal size assignments for global and partition dispatchers in the absence of admission control. The horizontal axis represents the task set utilization and the vertical axis represents the miss ratio. In the absence of admission control, a task might be scheduled under the thermal constraint, but it will violate the timing constraint. In this experiment, we compared the miss ratios, which is the number of task sets violating the timing constraint, of *TS-FF* and *TS-G* without applying admission control. In each subfigure, the figure at the top presents all the hot tasks in the task set, and the figure below presents all the cool tasks. In this experiment, we compared the thermal size assignments *Avg, Ba*, and *Pro* between *TS-FF* and *TS-G*.

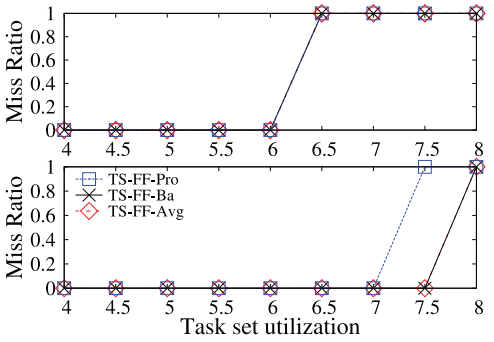
Fig. 5 shows the miss ratios in *TS-G* and *TS-FF*. The percentage of task sets that missed their deadlines is 4%, 6%, and 22% when all the tasks in the task sets are hot tasks with a total utilization of 6 for *TS-G-Pro, TS-G-Avg*, and *TS-G-Ba*, respectively. In the absence of admission control, *TS-G-Pro* assigns a large thermal size to the bottom core for scheduling more hot tasks with high utilization, leading to a low miss ratio. In the absence of hot tasks, *TS-G-Ba* outperforms *TS-G-Avg* and *TS-G-Pro* because all tasks are cool tasks and can be executed at the same frequency without the thermal problem, which is faced for hot tasks. In *TS-G*, although a core might have sufficient thermal size to execute a hot task, it might execute a cool task with a shorter deadline, leading to a longer waiting time for the hot task. In the absence of admission control, the performance gap between *TS-G* and *TS-FF* is smaller. All task sets are unschedulable in *TS-FF* and *TS-G* when the utilization is greater than 8.

As shown in Fig. 6, when the number of stacked layers increases from two to four, the total thermal size is shared by four layers. Therefore, the thermal size in each core of a 16-core four-layer chip is smaller than that in an 8-core two-layer chip. The thermal sizes are exhausted when the task set utilization increases, and finally, thermal size starvation occurs. In this study, *TS-FF-Pro* and *TS-G-Pro* yielded lower miss ratios compared with the other thermal size assignments (Fig. 6(a) and (b)). This is because *Pro* reserves larger thermal sizes for the cores closer to the heat sink, and these cores might offer greater schedulability for the assigned tasks. In the next experiment, we evaluate the schedulability of partition and global schemas with admission control.

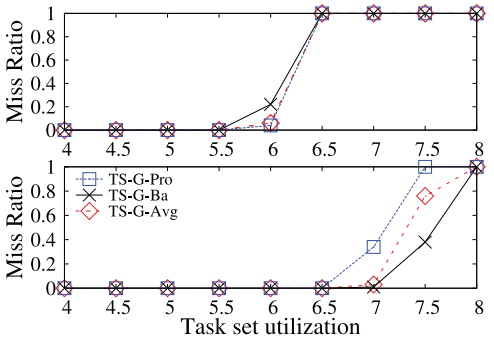
4.2.2. Comparison of thermal size assignments in the presence of admission control

Here, the experiment related to the schedulability of the thermal size assignment is discussed. The horizontal axis represents the task set utilization and the vertical axis represents the achieved ratio. When the proposed admission control is implemented, an unschedulable task can be rejected but the remaining tasks in the given set can be executed without violating the timing and thermal constraints.

The platform was set as an 8-core two-layer multicore chip (Fig. 7), and the achieved ratio for each assignment was evaluated. As shown in Fig. 7(a), all thermal size assignments yielded similar achieved ratios in the partition schema *TS-FF(AD)*. The reason is that the assigned tasks were executed on the respective cores without run-time task migration. By contrast, run-time scheduling was unpredictable in the global schema *TS-G(AD)*. When run-time task migration was considered, the thermal size assignment *Ba* led to a

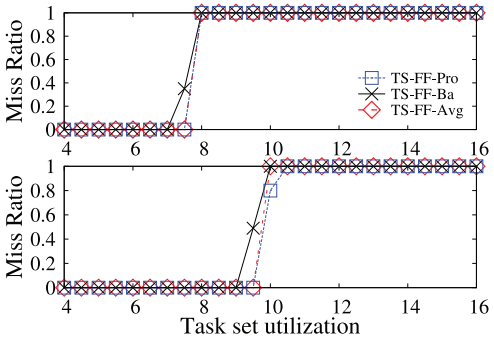


(a) Miss ratio for *TS-FF* (top side: 100% hot task, bottom side: 0% hot task)

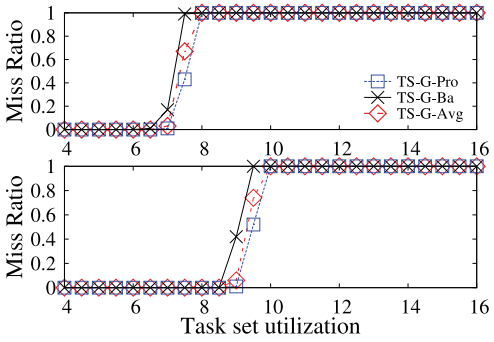


(b) Miss ratio for *TS-G* (top side: 100% hot task, bottom side: 0% hot task)

Fig. 5. Miss ratio of varied thermal size assignment in an 8-core two-layer chip.

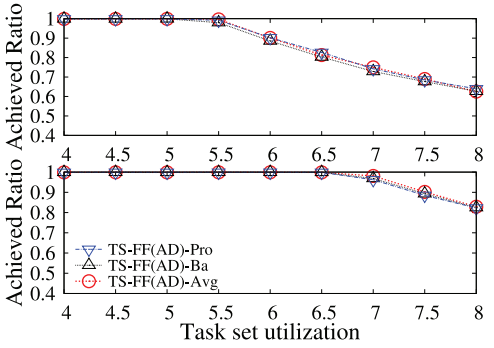


(a) Miss ratio for *TS-FF* (top side: 100% hot task, bottom side: 0% hot task)

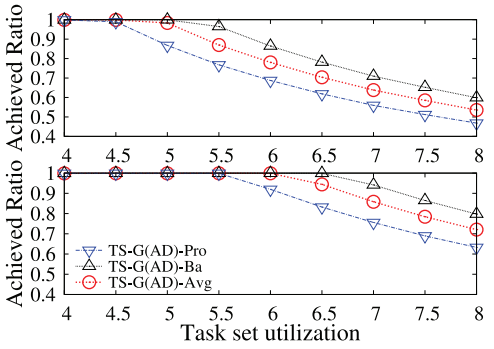


(b) Miss ratio for *TS-G* (top side: 100% hot task, bottom side: 0% hot task)

Fig. 6. Miss ratio of varied thermal size assignment in a 16-core four-layer chip.



(a) Achieved ratio for *TS-FF* (top side: 100% hot task, bottom side: 0% hot task)

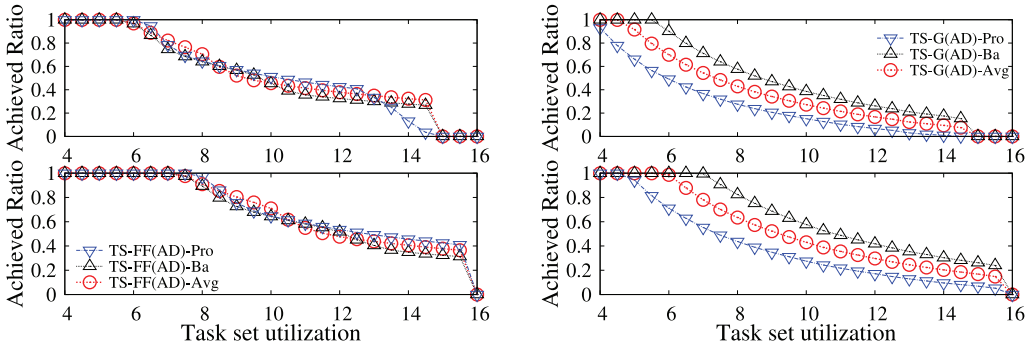


(b) Achieved ratio for *TS-G* (top side: 100% hot task, bottom side: 0% hot task)

Fig. 7. Achieved ratio of varied thermal size assignment in an 8-core two-layer chip.

higher achieved ratio compared with the other assignments because it prevented the execution time of a hot task from increasing drastically when the hot task was transferred to the top core (Fig. 7(b)) (i.e., for *Ba*, the thermal size of the top cores was larger than those for *Avg* and *Pro*).

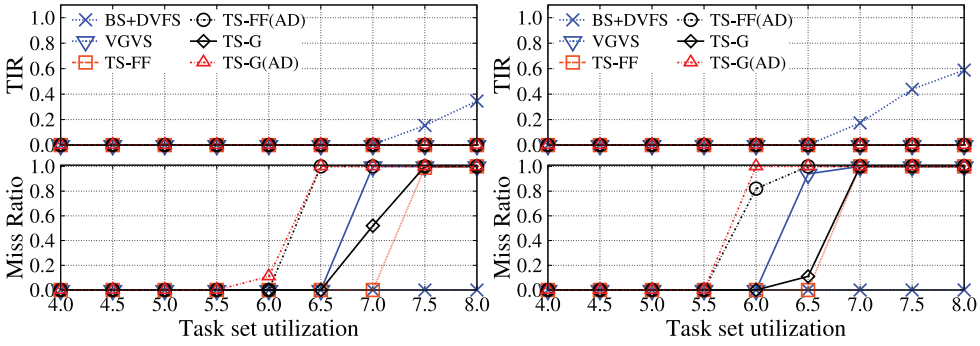
When the number of stacked layers was increased from two to four and the number of cores was increased to 16, as shown in Fig. 8, the achieved ratios in all thermal size assignments decreased. The reason is that the thermal size of each layer decreased with an increase in the number of stacked layers. The thermal sizes decreased when the task set utilization increased, and finally, thermal size starvation occurred, i.e., the thermal size for a core is too small to schedule a task. Thermal size starvation occurred when the task set utilization exceeded 14.5 and 15.5 for hot task ratios of 0% and 100% (Fig. 8(a) and (b)), respectively, and the increase in task set utilization was too high for the core with the smallest thermal size. In such a case, the thermal size assignment *Pro* showed higher performance compared with the others for *TS-FF(AD)*. This is because *Pro* reserved a larger thermal size for hot tasks for avoiding thermal size starvation. However, in the global schema *TS-G(AD)*, *Pro* showed poor performance because small thermal size assignments could not pass admission control and *Pro* contained the smallest thermal size among all thermal size assignments. The thermal size assignment *Ba* yielded a higher achieved ratio than those of *Avg* and *Pro* for *TS-G(AD)* because the smallest thermal size in *Ba* was larger than those in *Avg* and *Pro*, thereby increasing the probability of passing admission control.



(a) Achieved ratio for *TS-FF* (top side: 100% hot task, bottom side: 0% hot task)

(b) Achieved ratio for *TS-G* (top side: 100% hot task, bottom side: 0% hot task)

Fig. 8. Achieved ratio of varied thermal size assignment in a 16-core four-layer chip.



(a) 25% hot tasks in the workload

(b) 50% hot tasks in the workload

Fig. 9. Effect of varied hot tasks in an 8-core two-layer chip.

As shown in Figs. 7 and 8, the achieved ratio of *TS-FF(AD)* was higher than that of *TS-G(AD)* by approximately 0.28 for *Pro*. This is because run-time scheduling is unpredictable in global schema and the pessimistic admission control leads to poor performance. Because of the increased probability of passing admission control, the achieved ratios of *TS-FF(AD)* and *TS-G(AD)* were similar for *Ba* (difference ≈ 0.05).

4.2.3. Feasibility verification

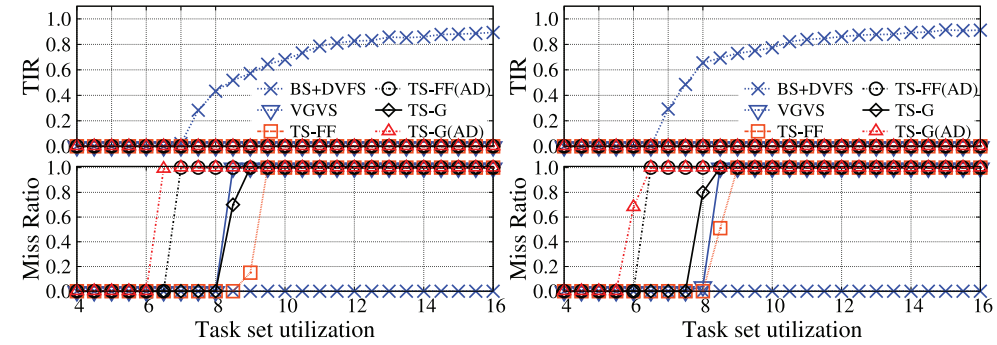
Here, we discuss the experiment performed to evaluate how the proposed approaches meet the thermal constraint and improve the schedulability compared with *BS+DVFS* and *VGVS*. The results in Section 4.2.2 show that in an 8-core two-layer chip, *Ba* shows performance superior to that of the other thermal size assignments in *TS-G* and *TS-G(AD)* when a hot task exists in the workload, whereas *Avg* shows the highest performance in *TS-FF* and *TS-FF(AD)*. Therefore, in the following experiments, we used *Ba* in *TS-G* and *TS-G(AD)*, and *Avg* was used in *TS-FF* and *TS-FF(AD)*.

Figs. 9 and 10 present the TIRs and the miss ratios of all approaches for the 8-core two-layer chip and 16-core four-layer chip, respectively. The horizontal axis represents the task set utilization, and the two vertical axes represent the TIR and miss ratio. The system violates the thermal constraint if the TIR is greater than zero.

As shown in Fig. 9(a), in *BS+DVFS*, when the hot task ratio is 25% and the task set utilization is greater than 6.5, the task set violates the thermal constraint because the cores cannot withstand the temperature generated from high utilization and hot tasks. This approach tends to accept all tasks and balance the temperature among the cores through the swapping of workloads between the hottest stack and the coolest stack. However, if the temperatures of most of the cores are high, *BS+DVFS* cannot guarantee the thermal constraint. More details of the PT in each core are presented in the next experiment. Unlike *BS+DVFS*, the *TS-FF, TS-FF(AD), TS-G*, and *TS-G(AD)* approaches do not execute an overheating task by using the thermal size constraint, as shown in the frequency determination of Algorithm 1. The frequency of each task was determined according to Eq. (7) and without violating the thermal constraint, by using the corresponding thermal size in the executing core ($P_{Core_j}(\tau_i, f_i) \leq \frac{TS_j}{R_j}$). However, the task might be refused when $u_i \times \frac{f_{max}}{f_i} > 1$, to meet the timing constraint. Therefore, the TIRs in *TS-FF, TS-FF(AD), TS-G*, and *TS-G(AD)* are all 0, but some tasks are refused when the task set utilization is greater than 7, 6, 6.5, and 5.5, respectively. In other words, the thermal-throttling server refuses tasks to guarantee the thermal constraint, but *BS+DVFS* executes all tasks and minimizes the temperature.

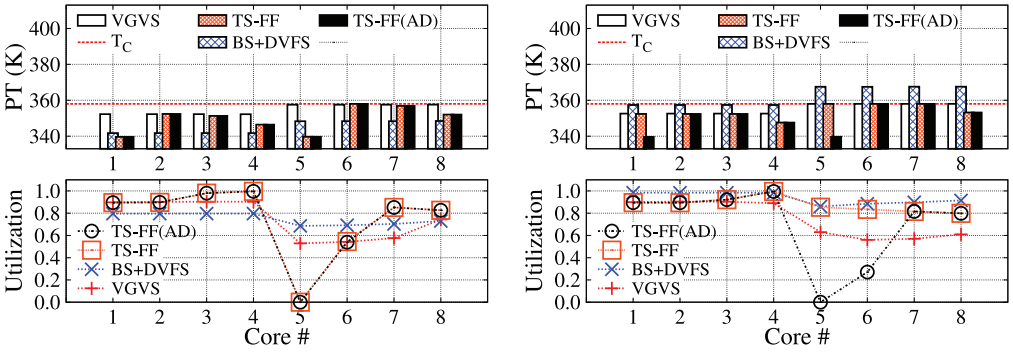
Unlike *BS+DVFS, VGVS* assigns a frequency to the core according to the peak power of the core to satisfy the thermal constraint. Moreover, to meet the timing constraints of tasks, the total utilization of a core does not exceed the assigned frequency. This results in the cooling utilization not being efficiently reserved by the identical frequency assignment of tasks in the same core. Although there is no thermal constraint violation (the TIRs are 0) in *VGVS*, all task sets are unschedulable when the task utilization is more than 6.5. In *TS-FF* and *TS-G*, all task sets are unschedulable when the task set utilization is greater than 7. Despite admission control, as shown in Theorem 1, not being applied to *TS-FF* and *TS-G*, these approaches guarantee the thermal constraint of tasks through Lemmas 2 and 1, respectively.

As shown in Fig. 9(b), when the hot task ratio in *BS+DVFS* is increased from 25% to 50%, the TIRs clearly increase. Furthermore, the achieved utilization decreases when the number of hot tasks increases, which is 6 in Fig. 9(b), without violating the thermal constraint. With an increase in the hot tasks, the number of feasible task sets decreases because the required cooling utilization increases. All

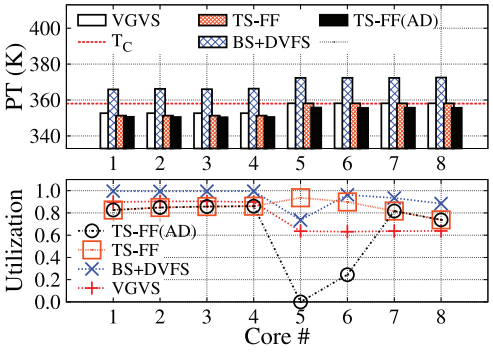


(a) 25% hot tasks in the workload (b) 50% hot tasks in the workload

Fig. 10. Effect of varied hot tasks in a 16-core four-layer chip.



(a) 25% hot tasks, U=6.0 (b) 25% hot tasks, U=7.5



(c) 50% hot tasks, U=7.5

Fig. 11. Effect of varied utilizations.

the task sets are unscheduable when the task set utilization is greater than 6, 6.5, 7, 7, 7 in *TS-G(AD), TS-FF(AD), TS-G, TS-FF*, and *VGVS*, respectively. However, *VGVS* achieved a miss ratio of approximately 95% when the task set utilization was 6.5. As shown in the aforementioned experimental results, for the same thermal constraints, *TS-FF* and *TS-G* achieved a higher schedulability and miss ratio compared with *BS+DVFS* and *VGVS*.

When the number of stacked layers is increased from two to four, and the number of cores is increased from 8 to 16, as shown in Fig. 10, the TIR in *BS+DVFS* clearly increases with the number of stacked layers. This is because all four cores in the same stack might be executed simultaneously, resulting in high temperature from the vertical heat flow. In *BS+DVFS*, the system overheats when the task set utilization is 7 and the hot task ratio is 25% (Fig. 10(a)). The performance gap between *TS-FF, TS-G*, and *VGVS* increases with the number of stacked layers for a hot task ratio of 25%. All task sets are unschedulable in *TS-FF, TS-G*, and *VGVS* at task set utilization of 9.5, 9, and 8.5, respectively. As shown in Fig. 10(b), when the hot task ratio increases from 25% to 50%, the miss ratio of *VGVS* is smaller than that of *TS-G* for a total utilization greater than or equal to 8. The reason is that the core might execute a cold task with a shorter deadline, leading to a longer waiting time for a hot task in the global dispatcher (*TS-G*). *VGVS* can be applied only to a partition dispatcher. Unlike *VGVS*, the thermal-throttling server can be used with both global dispatcher and partition dispatcher. When the partition schema is applied, the proposed thermal-throttling server (*TS-FF*) outperforms *VGVS*, even for a larger number of hot tasks or stacked layers. All task sets are unschedulable in *TS-FF* and *VGVS* when the task set utilization is 9 and 8.5 with hot tasks accounting for 50% of the tasks in the task set, respectively. This experiment showed that the thermal-throttling server can not only effectively manage tasks without violating the thermal constraint but also achieve superior schedulability.

4.2.4. Temperature distribution

For an 8-core two-layer architecture, Fig. 11(a) and Fig. 11(b) show the PTs and scheduled utilization of each core for all approaches

when the total utilization of a given task set is 6.0 and 7.5, respectively. In Fig. 11, the hot task ratio is 25% and the thermal constraint T_c is 358 K. The two vertical axes represent the PT and scheduled utilization of each core. The horizontal axis represents the index of the cores. The approaches *TS-G* and *TS-G(AD)* were not considered in this experiment because the scheduled utilization of each core was dynamically changed during run-time in global scheduling.

As shown in Fig. 11(a), when the hot task ratio of the task set is 25% and the task set utilization is 6.0, all approaches do not violate the thermal constraint and timing constraint. *BS+DVFS* balance the temperature among cores, as shown in Fig. 11(a). Although the PT among cores in *TS-FF* and *TS-FF(AD)* is higher than that in *BS+DVFS*, the proposed *TS-FF* and *TS-FF(AD)* never violate the thermal constraint. The scheduled utilization is not balanced between cores in *TS-FF* and *TS-FF(AD)* because of the effect of the first-fit partition. *VGVS* involves a higher temperature than *TS-FF* and *TS-FF(AD)* at $Core_1$ to $Core_4$ because it assigns high frequencies to the bottom cores. Consequently, the utilization of the bottom cores is higher than that of the top cores.

In *BS+DVFS*, when the total utilization of a given task set is increased, as shown in Fig. 11(b), the task set violates the thermal constraint. Furthermore, in the absence of appropriate admission control for thermal management, all the tasks in the given task set are assigned to the cores because the task set meets the timing constraint. Although *BS+DVFS* balances the workload and temperature among the cores, it violates the thermal constraint when the total utilization is increased. For an increase in the utilization, *TS-FF* achieved higher utilization in each core compared with *VGVS* because *TS-FF* effectively reserves cooling utilization for each core. A comparison of Fig. 11(b) and (c) shows that the PTs in *BS+DVFS* increase considerably with an increase in the number of hot tasks. However, in *TS-FF* and *TS-FF(AD)*, all the cores sacrifice more tasks to reserve more cooling utilization. In *TS-FF* and *TS-FF(AD)*, in the presence of appropriate admission control for thermal and timing constraints and when unscheduled tasks are present in the system, the total scheduled utilization of the given task set are 7.01 and 5.59, respectively (Fig. 11(b)). Because they remove unfeasible tasks, *TS-FF* and *TS-FF(AD)* never violate the timing and thermal constraints, as shown in Fig. 11.

5. Conclusion

In this study, we proposed a novel thermal-aware real-time scheduling framework for 3D multicore chips to achieve an effective trade-off between system temperature and task schedulability. We analyzed the thermal conduction from vertically aligned cores in 3D stacked multicore chips, and proposed the concept of thermal size and the manipulation of its assignment as a way to manage the heat generated by task executions. In contrast to off-line optimization approaches, the thermal-throttling server was first proposed to manage the heat generated by task executions by switching the core's frequency according to the assigned thermal size and run-time task information. A thermal-throttling dispatcher was then presented to enable thermal awareness in well-known real-time dispatchers, i.e., First-Fit and Global, for run-time task dispatching to cores. In order to ensure that all the task executions satisfied the timing and thermal constraints, an admission control for the proposed framework was also derived. Because of the lack of a 3D multicore thermal simulator, in this study, the performance of the proposed thermal-throttling server was evaluated by conducting a series of experiments involving synthesized workloads, and encouraging results regarding the scheduling capability and the prevention of overheating were obtained. In future research, we will continue to investigate thermal-aware real-time scheduling with more complex system architectures and thermal modes, and to develop a 3D multicore simulator for evaluating such systems. Further research and more experiences along these lines can enable the development of numerous mobile system designs.

Acknowledgments

This work is supported in part by a grant from MOST program MOST 102-2221-E-011-058 and part by a grant from the MOST program MOST 103-2221-E-011-111-MY3.

References

Austin, T., Larson, E., Ernst, D., 2002. Simplescalar: an infrastructure for computer system modeling. Computer 35 (2), 59–67.
Aydin, H., Melhem, R., Mossé, D., Mejía-Alvarez, P., 2001. Determining optimal processor speeds for periodic real-time tasks with different power characteristics. In: Proceedings of the Euromicro Conference on Real-Time Systems, pp. 225–232.
Chantem, T., Hu, X.S., Dick, R.P., 2011. Temperature-aware scheduling and assignment for hard real-time applications on MPSoCs. IEEE Trans. Very Large Scale Integr. (VLSI) Syst. 19 (10), 1884–1897.
Chaturvedi, V., Huang, H., Quan, G., 2010. Leakage aware scheduling on maximum temperature minimization for periodic hard real-time systems. In: Proceedings of the IEEE International Conference on Computer and Information Technology, pp. 1802–1809.
Chen, J.-J., Wang, S., Thiele, L., 2009a. Proactive speed scheduling for real-time tasks under thermal constraints. In: Proceedings of the IEEE Real-Time and Embedded Technology and Application Symposium, pp. 141–150.
Chen, J.-J., Wang, S., Thiele, L., 2009b. Proactive speed scheduling for real-time tasks under thermal constraints. In: Proceedings of the IEEE Real-Time and Embedded Technology and Application Symposium, pp. 141–150.
Chou, C.-H., Belviranli, M.E., Bhuyan, L.N., 2013. Thermal prediction and scheduling of network applications on multicore processors. In: Proceedings of the ninth ACM/IEEE Symposium on Architectures for Networking and Communications Systems. IEEE Press, pp. 115–116.
Chrobak, M., Durr, C., Hurand, M., Robert, J., 2011. Algorithms for temperature-aware task scheduling in microprocessor systems. Sustainable Comput. Inf. Syst. 1 (3), 241–247.
Chu, H.-H., Kao, Y.-C., Chen, Y.-S., 2015. Adaptive thermal-aware task scheduling for multi-core systems. J. Syst. Software 99, 155–174.
Cong, J., Luo, G., Shi, Y., 2011. Thermal-aware cell and through-silicon-via co-placement for 3D ICs. In: Proceedings of the Design Automation Conference. ACM, pp. 670–675.
Coskun, A.K., Ayala, J.L., Atienza, D., Rosing, T.S., Leblebici, Y., 2009. Dynamic thermal management in 3D multicore architectures. In: Proceedings of the Design, Automation & Test in Europe Conference & Exhibition, pp. 1410–1415.
Dreslinski, R.G., Fick, D., Giridhar, B., Kim, G., Seo, S., Fojtik, M., Satpathy, S., Lee, Y., Kim, D., Liu, N., Wieckowski, M., Chen, G., Sylvester, D., Blaauw, D., Mudge, T., 2013. Centip3De: a many-core prototype exploring 3D integration and near-threshold computing. Commun. ACM 56 (11), 97–104.
Fisher, N., Chen, J.-J., Wang, S., Thiele, L., 2009. Thermal-aware global real-time scheduling on multicore systems. In: Proceedings of the IEEE Real-Time and Embedded Technology and Application Symposium, pp. 131–140.
Hameed, F., Faruque, M., Henkel, J., 2011. Dynamic thermal management in 3D multicore architecture through run-time adaptation. In: Proceedings of the Design, Automation & Test in Europe Conference & Exhibition (DATE), pp. 1–6.
Healy, M.B., Athikulwongse, K., Goel, R., Hossain, M.M., Kim, D.H., Lee, Y.-J., Lewis, D.L., Lin, T.-W., Liu, C., Jung, M., et al., 2010. Design and analysis of 3D-maps: a many-core 3D processor with stacked memory. In: Proceedings of IEEE Custom Integrated Circuits Conference, pp. 1–4.
HotSpot. Technical Report. Univ. of Virginia Fund for Excellence in Science and Engineering. 2010. http://lava.cs.virginia.edu/HotSpot/.
Huang, H., Chaturvedi, V., Quan, G., Fan, J., Qiu, M., 2014. Throughput maximization for periodic real-time systems under the maximal temperature constraint. ACM Trans. Embedded Comput. Syst. (TECS) 13 (2s), 70.
Jejurikar, R., Pereira, C., Gupta, R.K., 2004. Leakage aware dynamic voltage scaling for real-time embedded systems. In: Proceedings of the annual ACM Design Automation Conference.
Jin, S., Wang, Y., Liu, T., 2015. On optimizing system energy of voltage–frequency island based 3-D multi-core SoCs under thermal constraints. Integration, VLSI J. 48, 36–45.
Kang, K., De Micheli, G., Lee, S., Kyung, C.-M., 2014. Temperature-aware runtime power management for chip-multiprocessors with 3-D stacked cache. In: 2014 15th International Symposium on Quality Electronic Design (ISQED). IEEE, pp. 163–170.
Kang, K., Kim, J., Yoo, S., Kyung, C.-M., 2011. Runtime power management of 3-D multicore architectures under peak power and temperature constraints. IEEE Trans. Comput. Aided Des. Integr. Circuits Syst. 30 (6), 905–918.
Lee, J., Ahn, J., Choi, K., Kang, K., 2015. THOR: orchestrated thermal management of cores and networks in 3D many-core architectures. In: 2015 20th Asia and South Pacific Design Automation Conference (ASP-DAC). IEEE, pp. 773–778.
Li, J., Qiu, M., Niu, J.-W., Yang, L.T., Zhu, Y., Ming, Z., 2013. Thermal-aware task scheduling in 3D chip multiprocessor with real-time constrained workloads. ACM Trans. Embedded Comput. Syst. 12 (2), 24.
Liao, C.-H., Wen, C.H.-P., Chakrabarty, K., 2015. An online thermal-constrained task scheduler for 3D multi-core processors. In: Proceedings of the Design, Automation & Test in Europe Conference & Exhibition. EDA Consortium, pp. 351–356.
Liao, W., He, L., Lepak, K.M., 2005. Temperature and supply voltage aware performance and power modeling at microarchitecture level. IEEE Trans. Comput. Aided Des. Integr. Circuits Syst. 24 (7), 1042–1053.

Liu, C.L., Layland, J.W., 1973. Scheduling algorithms for multiprogramming in a hard real-time environment. J. ACM (JACM) 20 (1), 46–61.
Liu, S., Zhang, J., Wu, Q., Qiu, Q., 2010. Thermal-aware job allocation and scheduling for three dimensional chip multiprocessor. In: Proceedings of the IEEE International Symposium on Quality Electronic Design, pp. 390–398.
Liu, Y., Dick, R.P., Shang, L., Yang, H., 2007. Accurate temperature-dependent integrated circuit leakage power estimation is easy. In: Proceedings of the Conference on Design, Automation and Test in Europe. EDA Consortium, pp. 1526–1531.
López, J.M., García, M., Diaz, J.L., Garcia, D.F., 2000. Worst-case utilization bound for EDF scheduling on real-time multiprocessor systems. In: Proceedings of the Euromicro Conference on Real-Time Systems, pp. 25–33.
Lopez, J.M., García, M., Diaz, J.L., Garcia, D.F., 2003. Utilization bounds for multiprocessor rate-monotonic scheduling. Real-Time Syst. 24 (1), 5–28.
Lung, C.-L., Ho, Y.-L., Kwai, D.-M., Chang, S.-C., 2011. Thermal-aware on-line task allocation for 3D multi-core processor throughput optimization. In: Proceedings of the Design, Automation & Test in Europe Conference & Exhibition (DATE), pp. 1–6.
Mizunuma, H., Lu, Y.-C., Yang, C.-L., 2013. Thermal coupling aware task migration using neighboring core search for many-core systems. In: 2013 International Symposium on VLSI Design, Automation, and Test (VLSI-DAT). IEEE, pp. 1–4.
PTScalar. Technical Report. University of California at Los Angeles. 2003. http://eda.ee.ucla.edu/PTscalar/.
Qualcomm Snapdragon Processor. Technical Report. Qualcomm Inc. http://www.qualcomm.com/chipsets/snapdragon.
Quan, G., Chaturvedi, V., 2010. Feasibility analysis for temperature-constraint hard real-time periodic tasks. IEEE Trans. Ind. Inf. 6 (3), 329–339.
Rabaey, J.M., Chandrakasan, A., Nikolic, B., 2002. Digital Integrated Circuits, second ed Prentice Hall.
Skadron, K., Abdelzaher, T., Stan, M., 2002. Control-theoretic techniques and thermal-RC modeling for accurate and localized dynamic thermal management. In: Proceedings of the International Symposium on High-Performance Computer Architecture, pp. 17–28.
Tsai, T.-H., Chen, Y.-S., 2012. Thermal-aware real-time task scheduling for three-dimensional multicore chip. In: Proceedings of the Annual ACM Symposium on Applied Computing, pp. 1618–1624.
Tsai, T.-H., Chen, Y.-S., 2014. A thermal-throttling server in 3d multicore chips. In: Proceedings of the 29th Annual ACM Symposium on Applied Computing. ACM, pp. 1425–1430.
Wilhelm, R., Engblom, J., Ermedahl, A., Holsti, N., Thesing, S., Whalley, D., Bernat, G., Ferdinand, C., Heckmann, R., Mitra, T., et al., 2008. The worst-case execution-time problem–overview of methods and survey of tools. ACM Trans. Embedded Comput. Syst. (TECS) 7 (3), 36.
Xu, R., Mossé, D., Melhem, R., 2005. Minimizing expected energy in real-time embedded systems. In: Proceedings of the ACM International Conference on Embedded Software, pp. 251–254.
Yang, Y., Zhu, C., Gu, Z., Shang, L., Dick, R.P., 2006. Adaptive multi-domain thermal modeling and analysis for integrated circuit synthesis and design. In: IEEE/ACM International Conference on Computer-Aided Design, 2006, ICCAD'06. IEEE.
Yao, F., Demers, A., Shenker, S., 1995. A scheduling model for reduced cpu energy. In: Proceedings of the Annual Symposium on Foundations of Computer Science, pp. 374–382.
Yun, B., Shin, K.G., Wang, S., 2013. Predicting thermal behavior for temperature management in time-critical multicore systems. In: 2013 IEEE 19th Real-Time and Embedded Technology and Applications Symposium (RTAS). IEEE, pp. 185–194.
Zapata, O.U.P., Alvarez, P.M., 2005. EDF and RM multiprocessor scheduling algorithms: Survey and performance evaluation. Seccion de Computacion Av. IPN 2508.
Zhou, X., Yang, J., Xu, Y., Zhang, Y., Zhao, J., 2010a. Thermal-aware task scheduling for 3D multicore processors. IEEE Trans. Parallel Distrib. Syst. (TPDS) 21 (1), 60–71.
Zhou, X., Yang, J., Xu, Y., Zhang, Y., Zhao, J., 2010b. Thermal-aware task scheduling for 3D multicore processors. IEEE Trans. Parallel Distrib. Syst. 21 (1), 60–71.
Zhu, C., Gu, Z., Shang, L., Dick, R., Joseph, R., 2008. Three-dimensional chip-multiprocessor run-time thermal management. IEEE Trans. Comput. Aided Des. Integr. Circuits Syst. 27 (8), 1479–1492.
Zhu, D., 2006. Reliability-aware dynamic energy management in dependable embedded real-time systems. In: Proceedings of the IEEE Real-Time and Embedded Technology and Applications Symposium, pp. 397–407.



Ting-Hao Tsai is currently working toward the PhD degree in the Department of Electrical Engineering, National Taiwan University of Science and Technology, and is supervised by Dr. Ya-Shu Chen. He received his BS degree in electrical engineering from National Taiwan University of Science and Technology at 2010. His research interests include operating systems, embedded systems, and real-time systems with thermal and energy issues.



Ya-Shu Chen joined Department of Electrical Engineering, National Taiwan University of Science and Technology, at August 2007. She currently serves as an Associate Professor. Ya-Shu Chen earned her BS degree in computer information and science at National Chiao-Tung University in 2001. Then, she studied in Department of Computer Science and Information Engineering, National Taiwan University, and was supervised by Prof. Tei-Wei Kuo. She successfully defended her master thesis and doctoral dissertation at 2003 and 2007, respectively. Her research interests include operating systems, embedded systems, and real-time systems with thermal and energy issues.